

Collaborative prioritization of architectural concerns

Lars Pareto^{a,*}, Anna Börjesson Sandberg^{b,1}, Peter Eriksson^{b,1}, Staffan Ehneborg^{b,1}

^a Chalmers University of Technology and University of Gothenburg, Gothenburg SE-412 96, Sweden

^b Ericsson AB, Stockholm 164 83, Sweden

ARTICLE INFO

Article history:

Received 10 January 2012

Received in revised form 18 April 2012

Accepted 23 April 2012

Available online 6 May 2012

Keywords:

Requirement prioritization

Software architecture documentation

Software architecture evolution

ABSTRACT

Efficient architecture work involves balancing the degree of architecture documentation with attention to needs, costs, agility and other factors. This paper presents a method for prioritizing architectural concerns in the presence of heterogeneous stakeholder groups in large organizations that need to evolve existing architecture. The method involves enquiry, analysis, and deliberation using collaborative and analytical techniques. Method outcomes are action principles directed to managers, and assessment of user needs directed to architects, along with evidence. The method results from 4 years of action research at Ericsson AB with the purpose of adding missing views to architecture documentation and removing superfluous ones. It is illustrated on a case where 29 senior engineers and managers within Ericsson prioritized 37 architectural concerns areas to arrive at 8 action principles, 5 prioritized improvement areas, and 24 improvement suggestions. Feedback from the organization is that the method has been effective in prioritizing architectural concerns, that data collection and analysis is more extensive compared to traditional prioritization practices, but that extensive analysis seems inevitable in architecture improvement work.

© 2012 Elsevier Inc. All rights reserved.

1. Introduction

Architecture documentation in large product development projects is, to an increasing degree, playing the role as a boundary object – a shared information resources used in different ways by different communities (Star and Griesemer, 1989). With contemporary practices, architecture documentation not only supports the daily work of architects and designers, but of requirement engineers, developers, testers, certification authorities, project-, product-, quality-managers and other roles whose work depends on architecture knowledge. Ideally, architecture documentation identifies the specific information needs of all of its users and provides architectural views that speak to each user community, so that overall work efficiency is optimized. In practice, being a taxing activity, architecture documentation is confined to the information needs most important – typically those with highest impact on project risk and cost (Fairbanks, 2010). Lower ranked needs are instead covered by informal coded communication (e.g., auxiliary design documents, source code, wikis, minutes, and correspondence) or uncoded communication (e.g., meetings, and phone calls), with the underlying – and quite plausible – assumption that, in

many cases, informal or uncoded communication is good enough, and sometimes even more cost effective than formal, coded communication.

However, prioritizing information needs with respect to importance is a non-trivial problem. First of all, information needs change throughout a product's lifecycle: some artefacts are critical in early phases, but lose importance in later phases, e.g., analysis models; information needs also change as new technologies, platforms, and processes come and go during a product's lifecycle. Secondly, information needs in large product development projects are so plentiful and heterogeneous that no single individual can comprehend all specific needs; for instance, telecom equipment projects involve hardware-, software-, mechanical-, and systems-engineering disciplines, each of which has several sub-disciplines, e.g., signal processing, multicore computing, fault tolerance and so on. In the third place, the degree to which specific roles contribute to the project's overall success, i.e., the economic importance of specific roles, is in most organizations not well understood and not easily quantifiable.

The objective of the research presented in this paper is a method for prioritization of architectural concerns held by architectural description stakeholders within enterprise-scale product developing organizations. By stakeholders, we mean, in this context, individuals with interests in architecture knowledge. By concerns we mean specific interests rooted in needs associated with a stakeholders role; by architectural descriptions, we mean architecture documentation composed out of models providing views from several viewpoints. By enterprise-scale, we mean that

* Corresponding author. Tel.: +46 31 772 6040.

E-mail addresses: pareto@chalmers.se (L. Pareto), anna.sandberg@ericsson.com (A.B. Sandberg), peter.r.eriksson@ericsson.com (P. Eriksson), staffan.ehneborg@ericsson.com (S. Ehneborg).

¹ Tel.: +46 31 719 0000.

a large number of distinct communities (typically more than 50) are involved in product development.

The research is rooted in a situation within Ericsson's business unit for telecom network equipment. In 2008, one of its design units experienced a need for stronger methods for *viewpoint engineering* – systematic application of engineering methods (e.g. requirements engineering, iterative prototyping, end-user validation, and managed evolution) to assure the fitness for purpose of architecture documentation in its usage situations. Viewpoint engineering had already been applied for several years, but the methods applied did not quite scale to the size of the supported organization, and the team responsible for the architecture documentation design had ran into several challenges, such as attending to the needs of all communities, communicating intended ways of working, getting acceptance from all stakeholder groups, and managing the evolution of the architecture documentation.

The root cause of these challenges first appeared to be the difficulty of eliciting all needs, but once that problem was solved, prioritization stood out as the paramount challenge. When concerns were documented, and put next to each other on paper, they just appeared as vast pool of good ideas (Armour, 2011), many of which would require research to realize, and of unclear relative importance. The problem was, essentially, a *requirements prioritization* problem (i.e., selection of which elicited candidate requirements that should be included in a product's requirements) with the architecture documentation, including infrastructure and processes for working with it, as the specified product.

Even though requirements prioritization is as a well-studied problem, supported by a wealth of methods (Berander, 2007), concern prioritization has turned out to have special characteristics calling for a new method: concern prioritization is an instance of requirements prioritization from the technical perspective, but not from the business and organizational perspectives. From a business perspective, the two problems differ considerably: traditional requirements prioritization methods consider business critical product requirements held by paying customers, whereas concern prioritization considers requirements on internal documentation held by internal, non-paying customers. From an organizational perspective, the problems differ too: development of innovative products with attention to customer or market needs is a core activity with high influence on organization; software architecture documentation, however, is a subsidiary activity that, if recognized at all by upper management, has little bearing on a company's organization and strategic investments. In other words, attention to internal efficiency, timeliness, and product quality is important for a company's business, but not as important as attention to market needs – which the histories of Motorola, Nokia and other companies sadly tell. This difference in business importance has far reaching consequences on how much process support, resources, and management attention the concern prioritization problem may get in an organization and in the end on how concern prioritization can be organized.

Established prioritization methods can be divided into *quantitative analytical methods*, e.g., AHP (Saaty, 1990), *qualitative analytical methods*, e.g., grouping (Berander and Andrews, 2005) and *collaborative methods* (Karydas and Gifun, 2006; Regnell et al., 2001). Methods differ with respect to degree of rationality, formality, granularity, ease of use, stakeholder involvement, scalability, duration, and other factors (Berander, 2007), thus choice of prioritization method is a contingency problem, where each method has its sweet zone. Choice of method inevitably involves trade-offs with respect to cost, validity, completeness, accuracy and other factors. The method sought in this paper is a *pragmatic method* that is feasible, yet good enough, for concern prioritization in large product developing organizations. The method presented is a mixed methods method that combines qualitative, quantitative, and analytical techniques

qualitative- and quantitative analytical techniques with collaborative techniques to improve architecture documentation in the direction of an overall business goal, with a balance between cost, completeness and other factors, adequate in an industrial context.

The remainder of the paper is organized as follows: we introduce a theoretical framework (Section 2) and our research design (Section 3); we present our out results which include a concern inventorying method (Section 4), a concern prioritization method (Section 6), and examples of applications of these methods in an industrial context (Sections 5 and 7). We then discuss validity of the results (Section 8), the method and its qualities (Section 9), related work (Section 10), and future work (Section 11). The paper ends with our conclusions (Section 12).

2. Theoretical framework

Methods research is served by a characterization of what problem the method solves, the context in which the method will be applied, and desirable qualities of the method. The following sections address these needs: Section 2.1 defines the concern prioritization problem; Section 2.2 describes a conceptual model for the prioritization method's context; Section 2.3 presents a quality model for prioritization methods.

2.1. Concern prioritization

Our view of architecture documentation is the one defined by IEEE 1471 – ISO 42010 (ANSI/IEEE Std, 2007); this standard for architectural descriptions recognizes relationships between a system, its stakeholders, its architecture, the description of the architecture, and a few auxiliary entities (environment, mission, and rationale). More importantly for this paper, it defines *architectural descriptions*, *concerns*, *views*, and the notion of *coverage*. In essence: an architectural description is a collection of products to document architecture; concerns are those interests which pertain to the system's development, its operation or any other aspects that are critical or otherwise important to one or more stakeholders; a view is a representation of a whole system from the perspective of a related set of concerns; a model is a formal structure that may participate in one or more views.

An architectural description is by nature selective: its views cover some concerns, but ignores other. What the architecture documentation covers depends on its purposes, what can be feasibly modeled, what is already present in other models, what is clearly visible in system's realization anyway, the costs of modeling, and on the benefits that modeling brings.

Concern prioritization is the act by which an organization selects which concerns the views of an architectural description should cover. For instance, consider the concern *algorithm modeling*. Some projects applying concern prioritization will find modeling of this concern un-necessary, as program code in a high level language is regarded a sufficiently abstract description of algorithms; other projects will find modeling of algorithms essentials, as incorrect behaviors may have disastrous effects (such as in automated trading).

2.2. Concern prioritization context

Factors influential to our concern prioritization method are given by the conceptual framework in Fig. 1. (See for example Miles and Huberman (1994) for an introduction to conceptual frameworks.) The central improvement object is the architectural description (AD). The AD provides views that do or do not cover architectural concerns important to stakeholders using it. Chief architects are stakeholders with technical power to define what

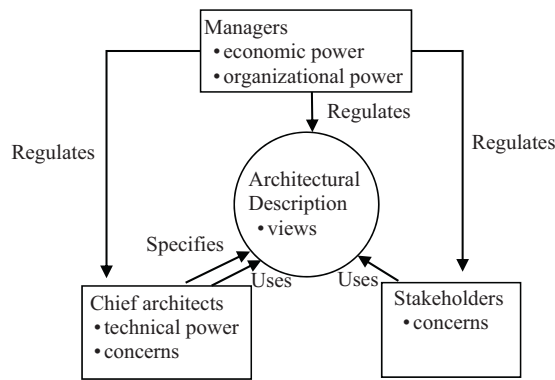


Fig. 1. The prioritization method's context.

Key: Conceptual framework notation (Miles and Huberman, 1994).

views the AD should contain, and that specify this. Managers regulate the ways in which AD is used by stakeholders, and have the economic and organizational power to evolve the AD and its associated ways of working.

2.3. Prioritization method qualities

Quality attributes of concern prioritization methods, of relevance to our design, include the following: *cost* is how resource demanding the application of the method is; *involvement* is the degree to which the method involves the stakeholders best prepared to make decisions; *interference* is the degree to which the method disturbs and taxes the organization; *prerequisites* is the degree to which the method requires special training in statistics, or other fields; *accessibility* is the degree to which method outcomes are accessible to decision makers; *rationality* is the degree to which decision making relies on rational judgment; *intuitivity* is the degree to which decision making relies on intuitive judgment; *correctness* is the degree to which the resulting prioritization can be trusted; *validity* is the degree to which method outcomes reflect the ideal prioritization.

A method's qualities vary with the situation. For instance, quantitative prioritization techniques excel on validity in situations where all factors of relevance to the prioritization, as well as their relative importance and dependencies, can be modeled and measured; they may lead to invalid results – and false confidence thanks to their scientific appeal – in situations where factors and relationships are not clearly understood or accurately measurable. Collaborative methods have high correctness if the parties have deep understanding of the domain, necessary social skills, and are committed to a common prioritization objective; they have low correctness in situations with uninitiated members, hidden agendas, or battles of prestige.

3. Research design

We approached our method design problem through action research: during a period of four years, the authors explored how discovery and prioritization of architectural concerns could be, feasibly, organized in a large product developing organization. The following sections motivate the choice of action research, as research strategy (Section 3.1), present the research site (Section 3.2), and outline the activities that led us to our proposed method (Section 3.3).

3.1. Motivation of action research as strategy

Action research (Lewin, 1946) is a research strategy in which the researchers are themselves involved in the context of the

investigation (rather than being passive observers). Our choice of action research was due to the organizational context of our study. Engagement of the studied site was conditioned on continuous alignment with local improvement goals and impact before theory building. The context of the study was complex, and learning about the context and the problem part of the research. The action research strategy is designed to support improvement under such conditions (Susman and Evered, 1978) and has proved itself efficient for software architecture improvement before (Christensen et al., 2008). At the site, action research was a preferred strategy with a good track record (Borjesson and Mathiassen, 2004; Sandberg et al., 2011).

3.2. Research site

Our research site was a software design division within Ericsson AB, a major supplier of telecommunication infrastructure equipment and services on several international markets. To motivate the need for our method, we will describe this division from the business, architecture, process and organizational perspectives (as of the BAPO model (Linden et al., 2007)) and its challenges.

3.2.1. Business

The division is responsible for the software part of a base station subsystem. Base stations are essential parts of mobile networks with thousands of units being needed to cover a country. (See e.g. Kaaranen (2005) for an introduction to mobile communication networks.) The operational lifetime of a base station is 15 years or more. Throughout a base station's lifecycle, customers (telecom operators or service providers) expect continuous product evolution, including new software features, hardware upgrades, and compatibility with evolving telecom standards. Change requests and trouble reports are frequent, and must be responded to promptly by any actor on the market. Frequent software upgrades are expected. Annual downtime must not exceed a few minutes, or penalties apply.

3.2.2. Architecture

The division studied uses model driven development (MDD). Ericsson has a long tradition in this: the company's use of software models dates back to 1967 and the AKE switch project (Jacobson, 1992). Ericsson contributed to development of SDL (Rockstrom, 1982) during the 1970s, and applied SDL extensively during the 1980s; use-case modeling was pioneered by Ericsson in the 1980s (Jacson, 1987); in the 1990s, Ericsson was an early adopter of RUP (Börjesson, 2006). Over the last decade, many projects have implemented model based software engineering and model driven development. Model based practices have gradually replaced document based practices which means that architecture documentation consist of blends of architecture descriptions and informal technical documentation (i.e. architecture documentation not relying on models).

The software developed by the division is embedded software, subject to real-time constraints (response time, throughput, and space bounds), compatibility constraints (with respect to real time operating systems and in-house developed platforms), and special run-time requirements (monitoring, configuration, upgrading, and rollbacks). The software runs on an in-house developed, multi-processor platform, which involves hardware (digital signal processors, field programmable gate arrays, and ordinary processors) as well as software (operating systems, protocol stacks, and device drivers).

The architecture documentation consists of a system model and several design models. The *system model* integrates software requirement-, design-, and implementation models and also artefacts on other forms such as informal diagrams, text documents,

spreadsheets, and text-based formal specifications. Integration of the system model and the design models is restricted to interfaces: software subsystems must adhere to interfaces formally specified by the system model, but may be realized with any MDD tool. (This is because not all MDD tools support digital signal processors, multicore processors, and other kinds of special hardware used in base stations, so variety is needed.) Hardware models are not integrated in the system model; rather, the system model includes descriptive models of hardware aspects relevant for software design. The system model is developed with due attention to the division's many processes (e.g., design-, development-, testing-, maintenance-, project management, and product documentation processes) and attention to needs to communicate with other divisions involved in the product development. Each division has their own design models, which must be held consistent with the system model.

Other kinds of models are used by units working within other engineering disciplines: in-house notations and domain specific notations of the kind prevalent in telecom standards are used by system engineers to define telecom networks, telecom nodes, and services; block diagrams and Mealy/Moore state machines are used by hardware engineers for design, implementation and description of hardware.

3.2.3. Process

In the division, requirements and design work proceeds along the lines of RUP, while implementation is done primarily using MDD, with occasional use of code centric development. Textual use cases and supplementary requirement specifications are used for software requirements modeling. UML is used for system design, system architecture work, software architecture work, detailed software design, software implementation, software testing, and systems testing. Where UML is used, it is often complemented with text based formal notations (e.g., signal and protocol specifications), with informal text-based documents (for aspects not easily expressed in standard UML such as non-functional requirements), and with tabular specifications (typically used for configuration information).

3.2.4. Organization

The studied division belongs to Ericsson's research and development (R&D) organization which, in addition to base stations, develops message routers, and various support nodes for several generations of mobile networks (2G, 3G, and 4G). The division is responsible for both software and hardware design, although the latter is restricted to special purpose boards. The hardware platform (a general purpose cabinet with boards and associated software) is not owned by the division, but developed and produced by a separate division, and used by the studied division and several other divisions: the platform must satisfy the needs of all divisions using it, i.e., not only the needs of the one considered in this study.

The division has a matrix organization in which, roughly, verticals are responsible for knowledge related to a specific product generation, and horizontal responsible for delivering and maintaining specific products. The design unit has approximately 100 engineers situated on a single location; the division is much larger and distributed over several locations. Roles with stake in

architecture documentation include system engineers, system architects, chief architects, software architects, designers, interface responsible, product responsible, developers, and product documentation responsible, unit testers, integration testers, system testers, team leaders, division managers and several other roles.

3.2.5. Challenges

In 2008, one project within the division, experienced consistency problems between the system model and their design model due to lack of architectural views at the subsystem level. Although UML-based architecture- and implementation design models were used by the unit, these were insufficiently aligned with the system model, which caused an abstraction gap left to designers to bridge; this was done in ad hoc ways and resulted in consistency and project predictability problems. A parallel project with longstanding documentation challenges (Pareto Boquist, 2006) reported on similar experiences. The shared experience of these two projects led to research on *concern coverage* (Pareto et al., 2009, 2011), and on the requirements on architecture descriptions serving as boundary objects between designers and system engineers (Pareto et al., 2010). These works recognized the importance of facilitating viewpoint engineering.

In 2009, a 3rd project implemented streamline development (Tomaszewski et al., 2008) – an in-house process for lean and agile software development in the large. Despite major modernizations to design documentation practices, this project experienced that “lack of a clear architecture” added significant lead time to projects, in particular for legacy systems. Methods for prioritizing architectural concerns – with the objective of removing superfluous ones and retaining only those most important for speed and throughput – was seen as a promising complement to streamline development, and led to the work described in this paper. In particular, the hope was that the method would help in increasing *speed* by pinpointing cases of over documentation.

3.3. Action research cycles

Five action research cycles concerned with viewpoint engineering were carried out (Fig. 2): one concerned with concern inventorying (Cycle₀); four with prioritization (Cycle_{1–4}). Cycles followed the canonical steps of action research described by Dickens and Watkins (1999): diagnosing, planning acting, and reflecting. Two cycles (Cycle₂ and Cycle₃) occurred in parallel, as two researchers focussed on different questions during the period. The durations of and questions addressed by each cycle are shown in Fig. 2, while activities and results of these are outlined in Table 1.

4. Results – concern inventorying method

Our concern prioritization method depends on availability of site specific taxonomy of architectural concerns. This section describes a method for construction of such taxonomy, which we refer to as *concern inventorying*. The method is an abridgement of the activities carried out in action research Cycle₀ described in Table 1. We first give an outline of the method (Section 4.1) and then proceed to specific techniques therein (Sections 4.2–4.7).

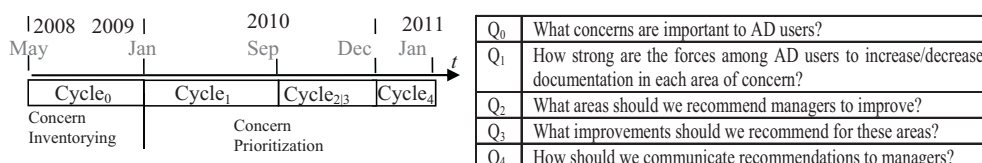


Fig. 2. Action research cycles (left) and the questions addressed in each cycle (right).

Table 1

Action research cycles and the steps taken in each cycle.

Step	Cycle₀ – concern inventorying
Diagnosing	Need to add missing views to AD.
Planning	Planned interview study to compare “mental views” among key stakeholders with the views provided by current AD.
Acting	Interviewed 16 stakeholders (architects, designers, testers) about missing views. Identified 142 concerns; structured these in taxonomy with 8 main areas and 37 sub areas. Assessed coverage of current AD; found it to be 26% of the concerns identified.
Reflecting	The set of concerns important to designers is vast and many faceted. What concerns AD should cover is controversial. Some roles suffer from under-documented concerns, while other produce documentation not used. Systematic viewpoint engineering (to identify, prioritize, and realize views for concerns) is needed. What business drivers that should drive prioritization are unclear: is it quality, cost, efficiency, automation, maintainability or some weighted combination of indicators?
Step	Cycle₁ – concern prioritization
Diagnosing	Need to understand what concerns to increase and decrease documentation for, to improve speed.
Planning	Planned and piloted a survey study to assess current AD-coverage of stakeholder concerns and whether concerns are perceived as under- or over-documented, given that the success indicator is speed. The pilot revealed several challenges in survey design: in limiting concern descriptions to “a few pages”; in keeping questions few and simple; in probing for concrete suggestions.
Acting	Executed survey. Invited 50 senior R&D engineers (architects, designers, line managers, project managers, and process engineers), out of which 29 responded: 68% from a >1000 person base station software project; 12% and 4% from two >100 person telecom support node projects; 12% and 4% from corporate concern and other functions. The survey was announced as difficult, but rewarding; respondents reported spending ½ a day on it. Analyzed survey answers using graphical statistical techniques. Studied change forces across projects only to find occasional differences. Inferred 37 hypothetical inferences from statistics. Presented statistics and inferences in a neutral way to respondents for feedback in a ½ day workshop. Documented responses in the form of reflections and observations. In the workshop, 35 out of 37 inferences were approved (after minor changes).
Reflecting	The survey and the workshop worked well in triggering reflections on current ways of working, in eliciting improvement ideas, and in prioritizing areas in need for improvement. To the dissatisfaction of some informants, the survey did not work so well in recognizing cases of over documentation: in the statistics, cases of over-documentation drowned in a generally positive attitude towards increasing documentation. We realized that our term “increase documentation” captured all kinds of changes to design documentation: improved documentation, different documentation, and even removal of redundancy from documentation. A surprising observation during data collection was hesitation among architects to take the survey: while designers contributed rich amounts of data, several architects found questions “difficult to make anything meaningful out of”. In the end, also the hesitant architects expressed interests in participating in analysis and deliberation activities, thus partly acknowledged the approach. A possible explanation is that of Farenhorst (2009): architects, in general, do not like the act of codification of architectural knowledge, while codified knowledge ranks #1 on their wish lists.
Step	Cycle₂ – concern prioritization
Diagnosing	Need to better understand what kinds of increases/decreases and other changes the informants had in mind. Need for a theory to analyze, compare, and discuss different kinds of changes to AD.
Planning	Planned a qualitative analysis of the answers to the open survey questions. These had been included to enquiry concrete improvement suggestions, but answers now stood out as a rich data source for theory building.
Acting	Coded and categorized survey answers to arrive at 144 concrete suggestions on how AD should be changed to increase speed. Structured suggested changes in a taxonomy recognizing 13 kinds of increases and 9 kinds of decreases. Observed that increase categories and decrease categories were different, and that taking averages of increase and decrease forces is misleading. Updated statistics to distinguish increases from decreases accordingly.
Reflecting	Increase- and decrease-forces among informants are of different nature, and should not be set against each other. Concern prioritization is an act without firm boundaries: one may bring out concerns in most (and least) need of attention, but to define precise thresholds of what to attend or not attend to is beyond what can be feasibly modeled in this context. The techniques employed are intuitive, do not require special training in statistics, and should be amenable to most managers.
Step	Cycle₃ – concern prioritization
Diagnosing	Need to cluster concerns in groups to make results communicable to upper managers.
Planning	Planned a qualitative study of commonalities of concern areas subject to high (or low) change forces.
Acting	Explored several notions for grouping concern areas in statistical plots. Developed a model recognizing both high and low forces to increase or decrease documentation. Defined a valuation function, and used it to define zones. Mapped concern areas to zones. Iteratively validated displays with insiders, and adjusted zones until a suitably large list of prioritized areas was found.
Reflecting	For results to lead to action, desired changes to AD need to be articulated at “executive level”: options should be few, concise, free from technical detail, and factual.
Step	Cycle₄ – concern prioritization
Diagnosing	Need to explain prioritization results to upper managers at a higher level of abstraction to make impact.
Planning	Identified variables that managers have intuition for; showed relationships between these and desired improvements.
Acting	Hypothesized 5 variables plausibly correlated to prioritized areas. Assessed variables using a survey sent to two key informants. Studied correlation between these variables and forces to increase or decrease documentation; found 3 out of 5 variables to be correlated. Formulated action principles on the basis of findings.
Reflecting	To be effective, all prioritization-activities and -results need to be stakeholder oriented. To engage designers, architects, and managers in improvement work, we need to express ourselves using different forms: designers are comfortable with domain vocabulary and statistical plots; architects expect and appreciate highly structured, refined, and traceable knowledge; upper managers are more likely to react to knowledge presented in the form of action principles. Engagement of all roles is needed: designers to give input on the needs; architects to select and realize or remove views; managers to make (the right) economic decisions and to approve change projects.

4.1. Concern inventorying overview

The concern inventorying method is outlined in Fig. 3. Concern inventorying addresses the fundamental question of what architectural concerns are important to the users of AD (Q_0). Its goal is a site-specific taxonomy of architectural concern: a hierarchical classification scheme that names and defines all architectural concerns important to the project using the site's vocabulary. The phase proceeds by iterative enquiry, analysis, and deliberation to construct and improve the taxonomy and ends with dissemination of results; it is ideally carried out by a small team of senior architects, engineers, and managers with complementary views on

the organization, and with experience from analysis work (which individuals in these roles typically have).

Specific techniques used for these activities are literature review, semi structured interviews, artefact collection (for enquiry), coding and categorization (for analysis), team- and informant workshops (for deliberation), which we now proceed to define.

4.2. Literature review

Concern inventorying is ideally done on the basis of existing taxonomies, thus a literature review is a natural first step in

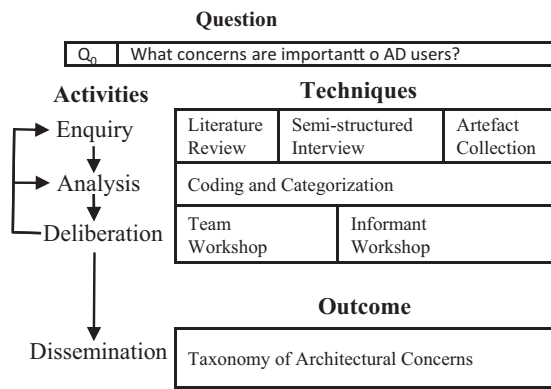


Fig. 3. Concern inventorying method.

inventorying. Sources to consider include industry standards, company internal glossaries, academic publications, and software architecture literature in general.

Qualities to consider when selecting taxonomies are their *domain* (business domain and product type), *organizational assumptions* (software processes, and tools used), *granularity* (level of details in the breakdown, level of detail in definitions), and *scope* (i.e., whether concerns also outside the area immediately “under the technology lamppost” (Medvidovic et al., 2007) are covered). Because most sources tend to have a focus on either technical, or organizational, or business concerns, synthesis of taxonomy from several sources is inevitable. The more representative, refined, and many-faceted the taxonomy is in the end, the more value it will bring to the prioritization process.

4.3. Semi structured interviews

Existing concern taxonomies are useful starting points in concern inventorying, but not effective theories: concerns are situational, and important concerns are not necessarily covered by available taxonomies. Semi-structured interviews (i.e., face to face inquiry about needs around open-ended questions) help

discovering and articulating new concerns not found during the literature review.

4.4. Artefact collection

Site specific concerns already covered by architecture documentation are best extracted from existing architecture documentation. Where interviews are effective instruments for recognizing concerns not addressed – frustrated individuals eagerly elaborate on these – they are less appropriate for recognizing concerns in place. These are readily available in the architecture documentation: in ADs, in AD-related documents, and in auxiliary design documents. Artefact collection is the act of gathering and selecting representative such data sources, by the help of stakeholders to obtain references.

Of particular importance, in gathering as well as selection, is attention to pictures of views and informal diagrams. Large product development projects can involve thousands of formal and informal diagrams, some of which are only available in modeling tools, other of which are only available in reports, presentations, educational materials, and other auxiliary sources. A broad scan for pictures in such sources, and selection of a representative subset of these pictures should be part of the artefact collection activity.

Of importance is also inclusion of non-pictorial formal artefacts, e.g., mathematical models, tabular models, tree-oriented models, metamodels, XML models, rules in textual, high-level, notations, or the kinds of textual models associated with formal methods. The degree to which AD is expressed using pictorial or non-pictorial formalisms varies from project to project, but no matter which modeling paradigm is the dominant one in a project, a team carrying out artefact collection should be cautious to consider also formalisms used less frequently.

Artefact collection does, however, not include gathering and selection of source code. Even though source code can be viewed as one form of architecture documentation, it encodes so many aspects other than architectural concerns (detailed design, data representation, algorithms, realization of fault handling, control flows, and platform details) on so many formats (programming, scripting, transformation, and testing languages), and is of such a large volume (millions of lines of code) that recognition of concerns in these source, at least with the analysis techniques of the proposed method, is infeasible.

4.5. Coding and categorization

Analysis, in concern inventorying, proceeds by the following steps. *Inductive coding* (Strauss and Corbin, 1998) of interview transcripts, wiki pages, design documentation, models, or written sources is used to reveal concerns that may otherwise be overlooked. *Deductive coding* (Strauss and Corbin, 1998) is used to see whether concerns of existing taxonomies are present or not. *Categorization* is used to organize concerns in a hierarchy of categories at increasing levels of abstraction from several perspectives.

Challenges in coding and categorization are the complexity of the domain analyzed, and that many concerns are crosscutting. Analysts must be able interpret sources from many engineering disciplines (requirements engineering, testing, product documentation, and so on), from many technical domains (radio signaling, internet protocols, power systems, and so on), as well as sources related to business-, process-, and organizational aspects. Further adding to the challenge, many concerns are crosscutting and naturally belong to several categories.

Although definition of ontology instead of taxonomy (i.e. modeling of not only concern category membership, but concern relationships in general) may appear tempting in such a situation,

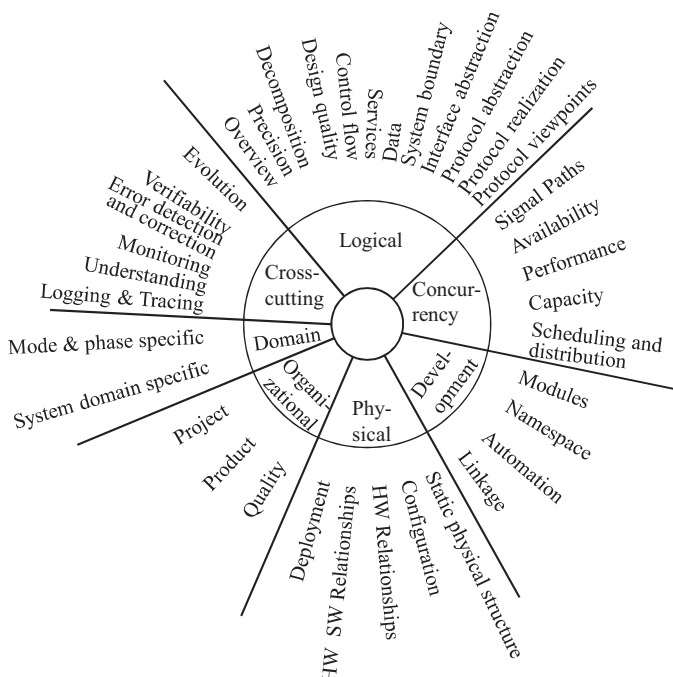


Fig. 4. Circular taxonomy tree from the Ericsson study.

our method does not involve ontology definition: first and foremost because it would manifold the resource demands and calendar time of a concerns inventorying project (and thus make it more difficult to get management approval for); secondly because, for purpose of concern prioritization, taxonomy is good enough (because, in the end, informants will look inside category definitions, at specific concerns, when quantifying their needs).

More feasible than ontology definition, is to apply the following strategy:

1. recognize cross-cutting concerns in the uppermost level of the hierarchy, next to other areas of concerns;
2. place concerns in the categories with their strongest home-base;
3. if no natural home-base can be found, revisit data sources to see if the concern recognized by the analyst can be split into several concerns that better fit the emerging taxonomy.

Concern taxonomy should be built with a tool that supports annotation and organization of pieces of text and pictures, where the support for pictures is needed to analyze models. Tool features and properties that have shown to be of particular importance are *coding support* (possibility to associate pieces of texts, or regions in diagrams, with the concerns they represent); *categorization support* (possibility to construct easily navigable hierarchies of concerns); *drill-down support* (possibility to navigate from a concern, to the raw data motivating its existence). Modern data analysis tools provide these features, but are – to their disadvantage – not widely known in engineering organizations, and assume specialist knowledge not held by typical managers and engineers. In such situations, semantic wikis (de Boer and van Vilet, 2011), which also satisfy the requirements above, may serve as an alternative.

4.6. Team and informant workshops

Team and informant workshops are deliberation activities, in which participants meet to carefully discuss all considerations of the emerging concern taxonomy: “Are all concerns there?”, “Have any important areas been missed?”, “Are all concerns relevant to the organization?”, “Are concerns expressed in a way that speaks to architects, managers, and developers in the organization?”, “Should concerns be renamed?”, “Are concern categories sensible?”, “Are concerns too fine grained, or coarse grained?”, “Are the concerns architectural concerns or something else?”, “Does concern classification make sense?”. A series of such sessions, interleaved with additional data collection and analysis work, to resolve issues found, eventually results in a taxonomy that the organization can accept as a valid model of the project’s all concerns.

Team and informant workshops also serve as a supplementary data collection technique. The deliberation sessions are recorded, and meeting notes and transcripts included as data sources. This has turned out to be a very effective technique, as deliberation involves in-depth discussions about the organization’s needs, the ways in which the AD serves these needs, the challenges or costs of realizing the needs, and the consequences of leaving the needs un-addressed.

In addition, team and informant workshops serve as ethnographic encounters, in which analysts assimilate the project’s situation, and what the root causes, underlying expressed needs, may be. The analysts’ understanding of the project’s processes, business situation, technical context, and organizational constraints have high impact on the analysis outcomes. Workshops, and reflections on such, are effective techniques to quickly build this understanding.

Finally, team and informant workshops are effective means of dissemination knowledge in the organization. During workshops, both team members and informants get ideas and inspiration for work they already do or could do. In a practice, when concern

prioritization eventually points at a problem needed to be solved, the improvement work may already have started, or the need even been resolved.

Team and informant workshops differ primarily in their organization. Team workshop are frequent (weekly), conducted by a small group that knows the goals and are accustomed to working together, and who invite special stakeholders as needed. Informant workshops are restricted to a few (2–3) bigger occasions, and involve more carefully prepared discussion materials (to communicate concerns effectively), advance planning (to involve important, but busy stakeholders), and stronger facilitation techniques (to manage the larger group, and risks associated with varying organizational powers among stakeholders and other social phenomena).

What workshop organization is most suitable depends on company culture. Proven organizations for informant workshops include the Delphi method or small-group group work. With the Delphi method, each participant prepares on the basis on a given set of questions; the prepared answers are presented in two or more rounds, and other participants are allowed to comment. A skilled and initiated meeting facilitator is needed to block non-constructive or diverging or irrelevant discussions, while allowing discussion that indeed reveal problems with the taxonomy discussed. In small-group group work, participants split to discuss and consolidate their preparations in thematic teams, then reunite for a bigger discussion.

In summary: recurrently feeding analysis results back to the organization through workshops reveals new areas and data sources previously overlooked, aligns terminology with vocabulary meaningful inside the organization, validates (or refutes) interpretations, and stimulates improvement work. Informant workshops can either be organized using the Delphi method or as small-group group work.

4.7. Taxonomy of architectural concerns

The outcome of the concern inventorying activity is, beside positive side effects on informants and analysts, the taxonomy. Canonical, generic deliverables (that take different forms and compositions depending on the tools used and organizational requirements) include the following:

4.7.1. Concerns catalogue

The concerns catalogue is a report listing and defining all architectural concerns, for the purpose of dissemination and data collection activities of later phases. An advantage of a plain, taxonomy free, listing of concerns and their definitions is that it draws the reader’s attention to the essentials – the set of fore lying concerns – rather than the organization of these concerns, which many engineers tend to give more attention than the concerns themselves. Concern prioritization is a pragmatic method, and the ambition of concerns inventorying to find taxonomy that gives enough structure to support prioritization of concerns. Many engineers (in particular those with research training) are, however, obsessed with perfection, and are quick to reject a report, or even the whole approach, if they find dissonances in the materials. Separation of concerns definitions and taxonomy thus makes acceptance building in the organization easier.

4.7.2. Category system

The category system report presents the category system on tree- or table form, defines all categories, and lists the concerns in each category (with reference to the catalogue). The purpose of the category system is to help stakeholders and analysts in navigating the concern set (by bundling similar concerns together into areas of concerns) and to provide stakeholders with contexts in which the typically brief concern names can be understood.

Table 2

Examples of concern definitions from the Ericsson study.

Concern 8: application decomposition

The base station considered provides two layered views of its resources that are externally visible for remote applications: the Operation and Maintenance (O&M) view; the Traffic Control view

Concern 9: approval status

Team leaders want better integration of information, in the system model, of what components have been approved to which degree.

Concern 112: upgrade

New versions of base station software are released frequently and remote upgrades common. Upgrades must be done with as little interference as possible (as downtime is very expensive for operators) and with support for rollback (should the upgrade go wrong). To support this, base stations have extensive upgrade architecture involving upgrade packages that bundle new software firmware- and description-modules for a node with an update script, and an upgrade engine that executes upgrade packages and monitors the upgrade process.

4.7.3. Circular taxonomy tree

The circular taxonomy tree is a pictorial presentation of the category system in the form of a circular taxonomy tree (which is a well-known display in biology). Although choice of display may appear as a trivial detail at first, it has methodological implications: inappropriate displays lead to large and complex displays that do not effectively communicate the results; large and complex displays scare stakeholders and decision makers (managers and chief architects) rather than triggering their curiosity; prioritization information will eventually be superimposed on the display, and some displays are more appropriate for this than others.

4.7.4. Knowledge base

The knowledge base consists of the coded data sources (transcripts, documents, and pictures), the category system, the analysts' memos (Strauss and Corbin, 1998), and tool specific data (queries and report generation templates). The knowledge base is input to the concerns prioritization activity as authoritative reference, and platform for evolving the taxonomy, should any concerns be unclear or controversial, or new concerns be discovered.

5. Example: concern inventorying outcomes of study at Ericsson

To make the method's objectives more concrete, the following section gives examples, from the study at Ericsson, of the concern inventorying outcomes. Notice that outcomes are influenced by particular choices of analysis tools and presentation formats, i.e., other instances of the method are likely to result in differently looking deliverables. We start by a description of the concerns catalogue (Section 5.1), and the concerns taxonomy on tabular (Section 5.2) as well as tabular form (Section 5.3). We also present the organization of the knowledge based resulting from concern prioritization, which becomes part of the evidence in an improvement recommendation.

5.1. Concerns catalogue

At the time of the prioritization study, the concerns catalogue consisted of 146 concerns, three of which are given in Table 2. We refer readers interested in the full set of concerns to our previous publications on these (Pareto et al., 2011; Pareto, 2010), which present an evolution of the catalogue of this paper with fewer concerns (114) and concern areas (25) but which covers the same concepts.

5.2. Category system

Table 3 is an excerpt of the category system: segments are major areas of concern (logical concerns, etc.); boxes are areas of

concern (overview, etc.); list items are specific concerns. Categories emerged from inductive categorization of concerns with attention to categories in Kruchten (1995). We refer readers to our past works (Pareto et al., 2011; Pareto, 2010) for details.

5.3. Circular taxonomy tree

Fig. 6 shows the circular taxonomy tree used in communication with stakeholders. For brevity, only the topmost categories were included, whereas stakeholders were referred to the concern catalogue for further details. Rendered versions of this display were used in several research project presentations and posters at the unit- and corporate level; they were extensively discussed by the unit's managers, when they first appeared, and became a trademark for the project.

5.4. Knowledge base

The essential elements of the knowledge base that resulted from concern inventorying are shown in Fig. 5. In addition to the data source described in Section 4 it includes text quotations and picture regions associated with codes, categories and subcategories of codes, set theoretic category queries, and memos. It also has functionality to export query results, category definitions, and code definitions on various data formats. Post processing of such exports (manual, automatic, and semi-automatic) was used to produce all displays in Sections 5.1–5.3. The tool used was ATLAS.ti (ATLAS, 2012), but any qualitative data analysis tool would have served.

6. Results – concerns prioritization method

The purpose of concern prioritization is to quantify stakeholder needs, collect improvement ideas, and to codify this information onto forms that managers and architects can relate to. This section describes a method for how this can be effectively done. The method is an abridgment of activities carried out in action research cycles Cycle_{1–4}. We start by explaining the rationale of the method (Section 6.1), and by giving an overview (Section 6.2) after which we proceed to describe the specific techniques used (Sections 6.3–6.13).

6.1. Concern prioritization methods rationale

The rationale of our concerns prioritization method (see Fig. 6) is to improve feedback from AD stakeholders to those with the economic, organizational, and technical power to improve it (managers and chief architects). Because of differences in perceptions and action repertoires, we must present this feedback differently to managers, who are more business and organizationally inclined and act through funds, priorities and missions, than to architects, who are more technically inclined and act through improvements of AD's, design processes, educations and tools. To architects, we want to present codified, traceable information on current needs and priorities; to managers we want to present actions they may take to improve architecture documentation along with evidence that recommendations can be trusted. By improving the feedback in this way, stakeholders will obtain better support for their tasks, which in turn results in increased business value.

6.2. Concern prioritization methods overview

Our concern prioritization method, which realizes this feedback loop, is outlined in Fig. 7. It is an abridgment of the action research Cycle_{1–4} outlined in Section 3.3; it proceeds by iteration over the same generic actions used for concern inventorying (enquiry,

Table 3
Examples of concern categories from the Ericsson study.

Logical concerns	
<i>Overview</i> - Complementary side perspectives - Capability anatomy - Functional areas - Major subsystems - Overviews - Tier dimensions and layers <i>Precision</i> - Overviews - Clean and precise visualization of requirements and behavior <i>Decomposition</i> - Application decomposition - Capability decomposition - Decomposition of functionality into technical subsystems - Load module structure - Platform as layer vs. platform as component - Process/subprocess decomposition - Service anatomy - System/subsystem decomposition <i>Design quality</i> - Clean and precise visualization of requirements and behavior - Coupling and cohesion among service realizations and capability realizations - System traces and system model consistency <i>Control flow</i> - Alternatives, conditions, and iterations in scenarios - Basic and alternate flows - Call graph - Component behavior	<i>Services</i> - RBS variables (parameters with Ericsson terminology) - Capabilities - Capability group filtering - Capability realization anatomy - Logical resources - System services <i>Data</i> - Class and capsule instantiation - Data – what's static, dynamic, or persistent - Locality (of reference) <i>System boundary</i> - Actors and use cases - Usage scenarios - External triggers <i>Interface abstraction</i> - Signal grouping and bundling - Allocation independent client-server interaction - Information flow routing between boards <i>Protocol abstraction</i> - Peer-to-peer signaling - Communication paths between abstract ports - External interface layering - Internal interface layering - Process interface inheritance <i>Protocol realization</i> - Layer-to-layer signaling - Data – coding of parameters sent in signals - Data – formal parameters in signals - Interaction between technical subsystems <i>Protocol viewpoints</i> - Service or use case realization - Allocation protocols - Protocols between technical subsystems
Organizational concerns	
<i>Project</i> - Baselines - Deliverables - Division of work - Planning - Responsibility <i>Product</i> - Corporate basic standard for information assets	<i>Quality</i> - Approval status - Protocol exemption (in behavioral descriptions) - Test coverage - Verifiability
Domain specific concerns	
<i>System domain specific</i> - Baseband use - Climate - Managed object (MAO/FRO/RO) domain - Power supply and distribution - RBS resource handling - Redundancy - Synchronization - Traffic control - Transaction handling	<i>Mode and phase specific</i> - Board configuration (main processors, device boards, auxiliary units) - Board loading - Connection establishment - Hardware start/restart (equipment restart) - Startup - Upgrade

analysis, deliberation, and dissemination) and a new *interpretation* activity. All techniques employed are different, except for the deliberation activities which have the same form.

Another difference is that several questions are incrementally addressed: the method sets out to assess the improvement forces among AD users (Q_1), introduces additional questions each iteration (Q_{2-4}) while refining the answers to questions of preceding iterations as knowledge grows. The techniques are described in the following.

6.3. Concern coverage survey

The concern coverage survey serves three purposes: it collects views on the current degree of documentation in specific areas; it collects opinions on whether to increase or decrease documentation in these areas; it collects suggestions of what to change therein. The use of surveys, rather than interviews, for data collection has several advantages: surveys reach out to larger populations; they are comparably quick and inexpensive; they support collection of

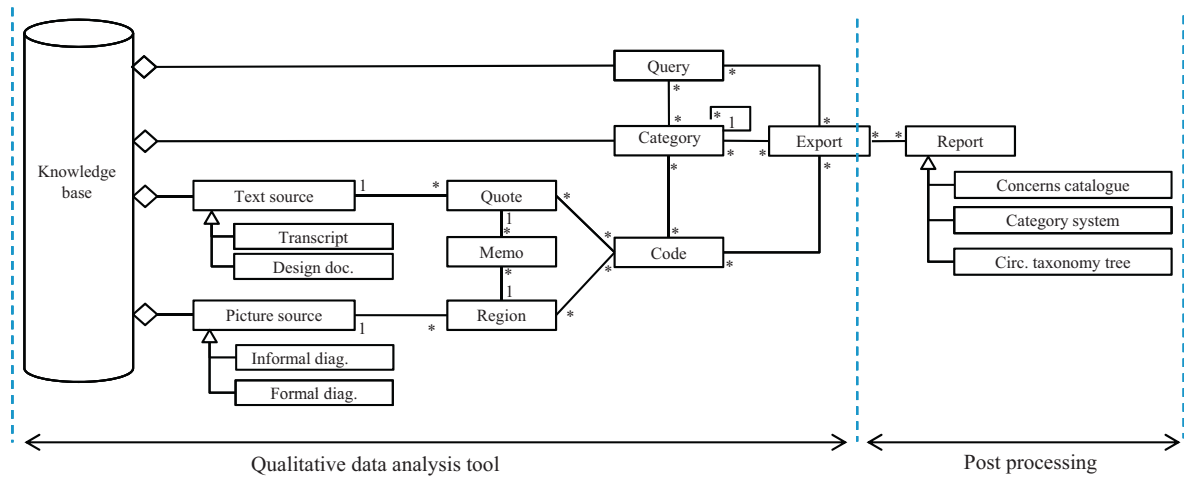


Fig. 5. Concern inventorying knowledge base (notation: UML).

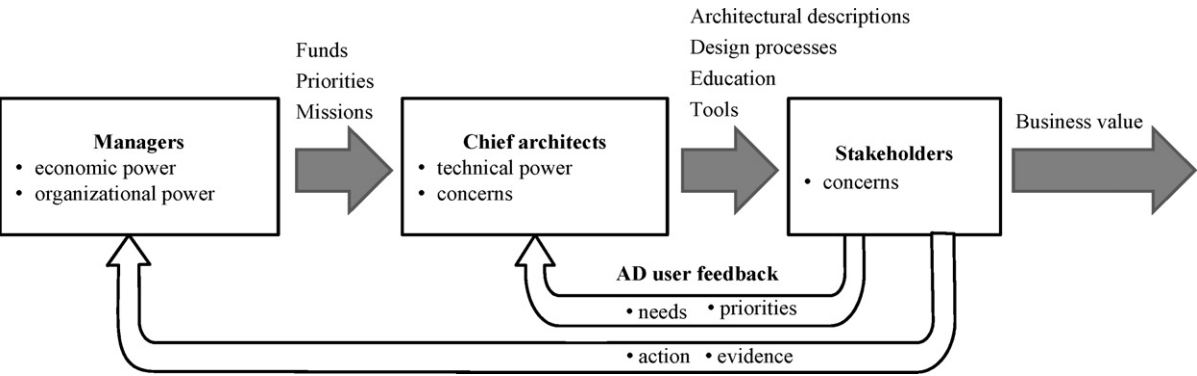


Fig. 6. Method rationale: improved feedback from AD users, to AD owners and maintainers.

quantitative as well as qualitative data; managers and developers are accustomed to surveys. The disadvantages of surveys over interviews – surveys are not as congenial and interactive as interviews may be – are, in our method, partly compensated for by informant workshops.

Generic survey questions and answering options are given in Table 4. The survey starts with enquiry about role and context. Its

core part is a set of questions for each area of concern, followed by questions for each major area for concern. The survey ends with reflection questions on the situation as a whole. Notice the relationships between the questions and the taxonomy's structure: questions relate to areas of concerns rather than specific concerns; for each area, respondents can elaborate on, or provide additional, specific concerns using open text input, which can also be done for

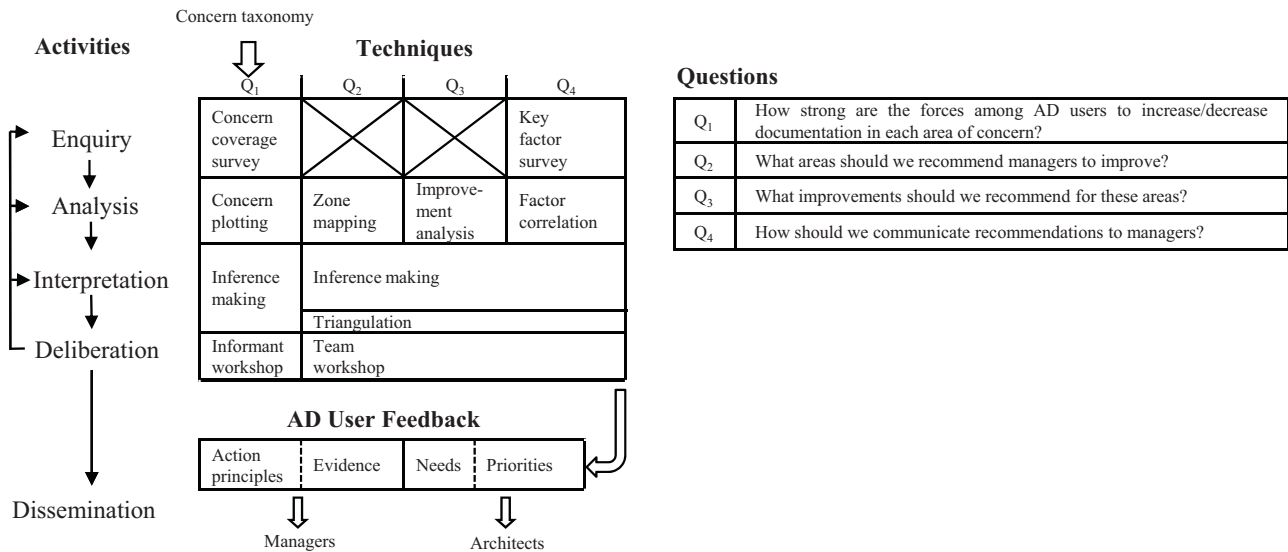


Fig. 7. Concerns prioritization activities, techniques, questions and outcomes.

Table 4

Generic survey questions to be asked for each concern area A, major area M, for the overall business goal G.

Questions about the informant	Options
What is your main role?	Manager developer other
Which product family do you primarily work with?	$P_1 \dots P_n$
Which project will you use as case when answering the questions?	Open text input
Questions for each area A	Options
How would you classify the present support in area A?	Not documented weakly documented scantily documented sufficiently documented well documented strongly documented don't understand
How should the support change if G is to be improved?	Essential to decrease desirable to decrease slightly advantageous to decrease slightly advantageous to increase desirable to increase essential to increase no intuition
What kind of increases/decreases of information would you like to see in area A? (For instance: more or less formality; more or less information content; addition of X; removal of Y; other changes.)	Open text input
Anything you would like to add in the area A as a whole?	Open text input
Questions for each major area M	Options
Any further comments of the major area M as a whole?	Open text input
Additional reflections on concern coverage regarding what is done well, and urgent improvement needs?	Open text input
Concluding reflection questions	Options
Considering architectural concerns and G, what do you think we do well within the company today?	Open text input
What do you think we urgently need to improve?	Open text input
I would also like to add the following regarding architecture and G:	Open text input

each major area and for architecture documentation in general. Also notice the implications of the taxonomy's structure on the questions: wide and few concern areas make the questions difficult to answer; narrow and plentiful areas make the survey unwieldy to answer. The Ericsson survey had 37 areas, and several informants reported on spending 4 h completing all questions; so 25–50 areas should be considered as practical limits.

Central to prioritization are the questions “How should the support [for area A] change if G is to be improved?”, where A is a concern area, and G is a *business goal* such as increased quality, reduced development costs, quicker time to market for new products, or increased development efficiency. Probing for improvement in the direction of a specific goal is important, because what supports fulfilment of one goal, may be a counterforce for the fulfilment of other, e.g., increased quality typically leads to either longer development time or additional costs. Without a clear goal, we will collect disparate improvement ideas aiming at possibly obsolete business goals. The business goal should be presented in an accessible way, and such that it cannot be missed: in the invitation, on the survey's first page, in the survey questions and in a separate survey guide.

The survey guide is a written document that includes an introduction to the survey and its purpose, a clear definition of the business goal, the taxonomy, and instructions for answering the questions. Conciseness of the survey guide is critical (max. 10 pages as a rule of thumb), as large volumes of texts and lengthy definitions have a repelling effects on informants. Leaving out the concerns catalogue (i.e. the most detailed level of the taxonomy)

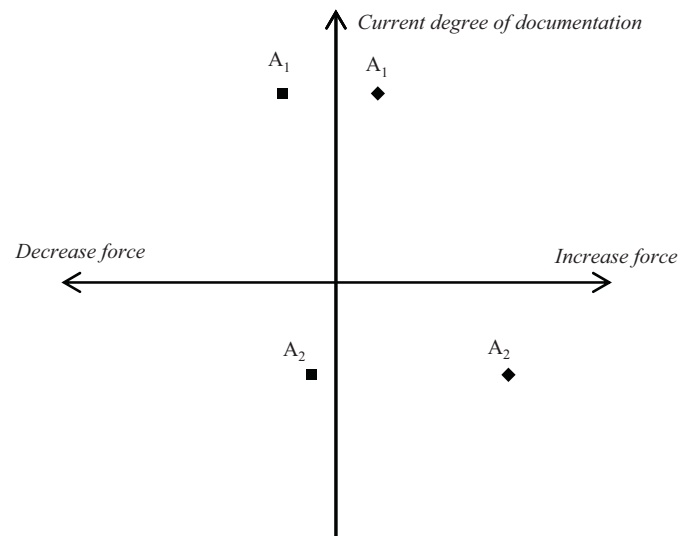


Fig. 8. A concern plot showing the increase- and decrease-forces and the current degrees of documentations for two areas of concern.

while providing a reference to a technical report describing it, is an effective strategy to compactness. This strategy works, because in a concerns catalogue relying on local terminology, carefully crafted concern names speak for themselves without lengthy definitions.

The concern coverage survey should be sent developers, managers and other stakeholders in architecture documentation, e.g. those involved with specification, design, realization, design documentation, product documentation, testing, certification, maintenance, evolution. Depending on business context, stakeholders may be internal or external. Informants should be senior, have longstanding experiences from architecture documentation work, from associated processes, and from the organization as a whole.

Surveys should be distributed and collected with a system supporting response tracking at the level of individuals. As most survey researchers have experienced, making informants to commit to and to actually deliver on long questionnaires, takes several reminders, negotiations, and social maneuvers (such as group pressure or appeal to everyone's best). Support for book-keeping of responses, directed notifications, and the opportunity to follow up with phone calls and visits is essential for the feasibility of the method. Several software packages and hosted services with such support are available on the market.

6.4. Concern plotting

A concern plot is a statistical display for visual comparison of three variables: the *current degree of documentation* of an area of concern A, the *force to increase*-, and the *force to decrease* documentation of A. It consists of an overlay of two scatter plots: one comparing current degree of documentation with decrease forces (shown along its leftwards pointing horizontal axis); another comparing current degree of documentation with increase forces (shown along its rightwards pointing horizontal axis). For instance, in the concern plot in Fig. 8 for the two areas A₁ and A₂, we can observe the following: A₁ is perceived as more documented than A₂; there are no strong forces to increase, neither to decrease the documentation of A₁; area A₂ is regarded as weakly documented; there is a strong force to increase documentation of A₂, but almost no force to decrease it. All in all, we see that any improvement efforts should prioritize A₂ over A₁ as A₂ is comparably under documented, and as there is almost consensus that documentation of A₂ should be increased.

The rationale behind concern plots is that prioritization decisions should take not only change forces into account, but also the current documentation maturity. This is for two reasons. The first reason is the *likelihood of reward*: improvement efforts for under-attended areas are likely to be more rewarding than for areas with longstanding attention. Scatter plots help managers and analysts in making such judgments. The second reason is that a psychological effect, *documentation greediness*, is at play: in the studied organization, we observed that concerns globally recognized as important in the organization, attracted attention even though their documentation was up to standard. Two-variable plots bring this effect, too, into the picture.

The rationale behind separating increase forces from decrease forces (rather than taking an average) is that the underlying phenomena quantified by these variables are not counterforces. Qualitative analysis of the suggestions in the survey's open questions has shown that suggestions to increase documentation in an area, and the suggestions to decrease documentation in the same area consider different matters, or sometimes the same matter phrased with a different polarity. For instance, what one informant expressed as "increased overview", other expressed as "decreased degree of detail". This implies that taking an average of all forces is a meaningless operation: strong forces to increase documentation in an area, and equally strong forces to decrease documentation of the same area do not imply that the situation is fine. On the contrary, this implies that design documentation in this area is *under-engineered* and deserves a closer look: the information that informants would like to see in the documentation is not there, but instead information that is not needed. By looking for large distance between increase and decrease forces, analysts can quickly spot areas in need for further attention, while at the same time judging whether the needed improvement would involve adding new information (which is comparably easy) or removing information (which is comparably difficult in projects of the studied kind), or whether documentation in the area needs major redesign (which is difficult, but sometimes necessary).

The variables used in a concern plot have the following definitions. Let C_1 , C_2 and C_3 be the number of respondents that find the documentation *sufficiently*-, *well*- or *strongly documented* in some area A ; let C_{-1} , C_{-2} , and C_{-3} be the numbers of those that find it *scantly*-, *weakly*-, or *un-documented*; further, let n be the total number of responses including "don't understand" answers. The current degree of documentation in area A , C_A , is defined as the following linear combination:

$$C_A = \frac{1}{n} \cdot \sum_{i=-3}^3 i \cdot C_i$$

The decrease force for area A , D_A and the increase force for area A , I_A , are defined similarly. Let D_1 , D_2 , D_3 be the number of respondents finding A *slightly advantageous*-, *desirable*- or *essential to decrease*; let I_1 , I_2 , I_3 be the number of respondents finding A *slightly advantageous*-, *desirable*-, or *essential to increase*; let n be the number of respondents for the area, including "no intuition" answers, then

$$D_A = \frac{1}{n} \cdot \sum_{i=1}^3 i \cdot D_i, \quad I_A = \frac{1}{n} \cdot \sum_{i=1}^3 i \cdot I_i$$

6.5. Inference making (Q_{1-4})

Inference making is a technique to turn *data displays* (concern plots and conventional statistical displays) or qualitative raw data into verbal hypotheses about the current situation, for the purpose of communicating the situation to managers, chief architects, and

the development organization as a whole. Although statistics may speak to initiated analyst teams and survey respondents, statistics is not sufficiently succinct and accessible for others, hence the need for verbalization. Inference making proceeds by collaborative hypothesis generation around statistical displays, qualitative data displays, or transcripts. Useful statistical displays are pie-charts, bar-charts, and histograms showing

- the understanding of concern areas among informants;
- non-understanding of concerns per role
- desires for change across roles;
- distribution of answers for each area of concern;
- distribution of informant roles, and
- population of analysis categories.

An example inference is given in Fig. 9. Recognizing that managers are fewer than developers among informants, while being over-represented among those not understanding the concerns $A_1, \dots, A_{11}$, we may plausibly assume that managers find this class of concerns difficult to understand (compared to developers and other roles, most of which understand them).

6.6. Informant workshop (Q_1)

The purpose of the informant workshop of the Q_1 iteration is to validate the statistical displays, validate inferences made from these, and to generate further hypotheses. It involves the analysis team (as presenters, discussion facilitators, and data collectors) and is open to all survey respondents. (In the Ericsson case, 50% of the informants were able to attend.) The workshop has two sessions. In the *interpretation session*, the statistical displays (distributed to informants in advance) are presented in an unbiased way, and discussed by the informants, while analysts take notes; analysts should be careful, during this session, not to inject opinion. In the *validation session*, analysts present their own hypotheses and their evidence while informants collectively comment, criticize, approve, or refute the hypotheses or the evidence. Notes-taking by the analysts, is used to document the response. By encouraging informants to make their own interpretations before presenting hypotheses, misinterpretations are more likely to be discovered.

The outcome of the informant workshop is data supporting the validity of some displays and some inferences made, and a list of issues for the others. These are fed into next iteration for further analysis and resolution. Because participating informants have strong intuitions about the problem, have studied the survey guide in detail, and spent several hours on thinking about the questions, the data collected in the workshop is (if the workshops is conducted well) of high quality and becomes a valuable data source for evidence building.

6.7. Zone mapping (Q_2)

Zone mapping is a supplementary graphical technique that helps analysts to derive improvement priorities from concern plots. A zone map (see Fig. 10) is a graphical representation of a function that maps point in the concern plot to the priorities DEC—, DEC—, DEC, OK, INC, INC+, INC++, where DEC— denotes "strong reasons to decrease", etcetera.

The zone map is divided in three basic zones: one for areas with an appropriate degree of documentation (OK zone); one for areas deserving increased documentation (INC zone), and another for areas deserving decreased documentation (DEC zone). The increase- and decrease zones further contain sub-zones recognizing priority ($\text{INC++} \subseteq \text{INC+} \subseteq \text{INC}$ and $\text{DEC—} \subseteq \text{DEC—} \subseteq \text{DEC}$).

Zone borders denote "height curves" of a surface defined by a two-variable function $f(x, y)$ with certain properties: f should be

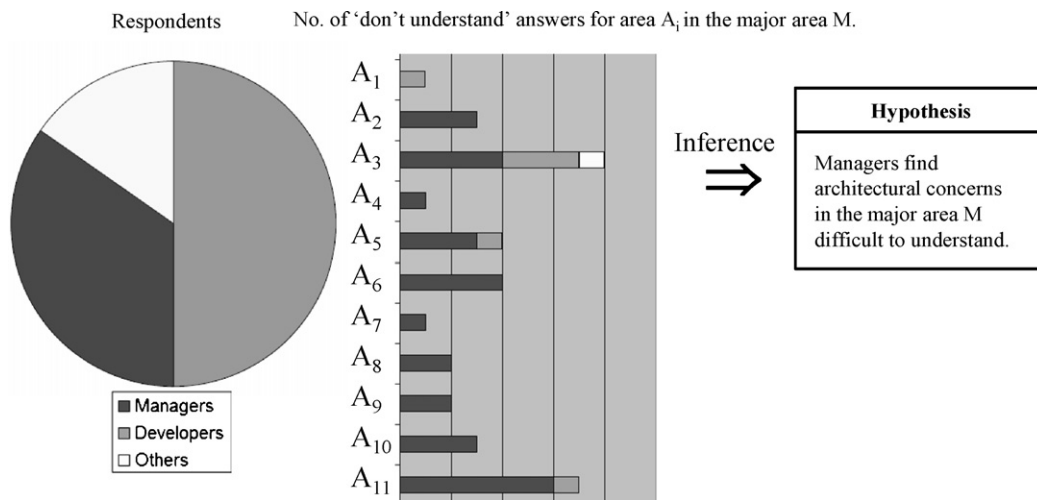


Fig. 9. Example inference from two displays.

continuous, strictly monotonic, and increasing in x and y ; if (x, y) is in the OK zone, then $f(x, y)$ should be 0. Intuitively, functions with these properties define a valley of adequately documented concerns surrounded by the slopes of concern increase- and decrease: the higher the altitude, the more attention the point deserves. Notice the curved form of the INC zone, which is due to the phenomena discussed in Section 6.4: the INC zone is tilted right because of the likelihood of reward effect; it bends off to the right to downplay documentation greediness. Similarly, the slope of the DEC zone is tilted because decreasing over documented concerns is better than further decreasing under documented ones; the DEC zone bends off to the left to “protect” weakly documented concerns from not being documented at all (which *extreme agilism*, the dual of documentation greediness, may lead to).

Also notice that the distance to the INC zone from the origin is larger than the distance to the DEC zone. This, too, is to compensate for documentation greediness. In the Ericsson study, recognized opportunities to increase documentation outnumbered recognized opportunities to decrease documentation, which results in weaker forces on the decrease side of the concern plot that on the increase side. This motivates a “handicap” for concerns in need for decrease, which the closeness of the zone gives.

Zone mapping is in itself straightforward. We give a concern area *high attention* if it falls either in an INC+(+) or DEC-(−) zone,

and *moderate attention* if it falls either in and INC or DEC zone, and no attention if it falls within an OK zone. This can be done simply by placing a transparent electronic image on top of the concern plots and then read off the priorities; more sophisticated is the use of a statistics software tool which allows coding of the surface and automated classification and ranking with respect to the function f .

6.8. Triangulation (Q_{2-4})

Triangulation is the use of several methods to double check the results. In our method, it is an implicit technique, yet with strong influence on results. As analysts work iteratively, with multiple data sources and methods, findings from later iterations will sometimes confirm and sometimes contradict the findings of earlier iterations. This forces analysts to revisit the data sources and the inferences drawn from these, or – if the contradiction cannot be resolved – to withdraw their hypotheses.

6.9. Team workshops

Team workshops are organized deliberation activities, in which analysts meet to critically discuss analysis outcomes so far. Guests

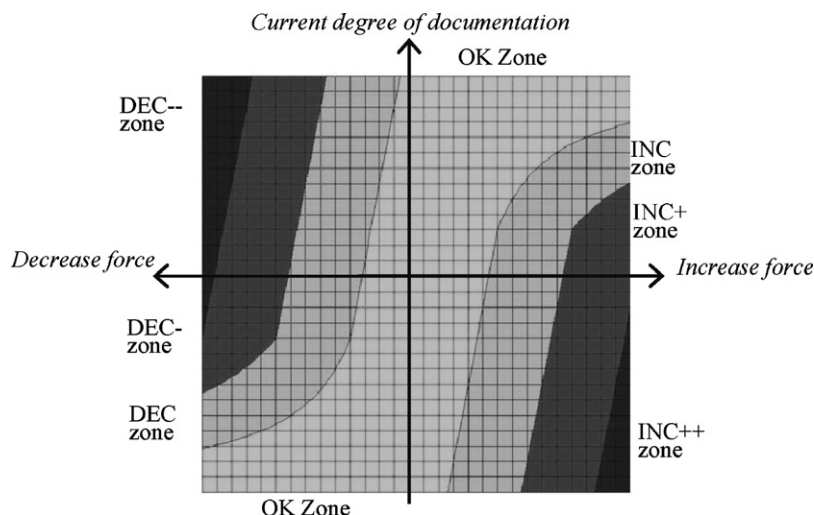


Fig. 10. Example of a zone map.

Table 5
Generic improvement actions for architectural descriptions. (Actions are empirically grounded.)

Kinds of decreases of AD	Kinds of increases of AD
Remove views not used	Add missing views
Reduce component granularity	Cover more cases with existing views
Reduce bureaucracy and formalities around documentation	Document more architecture layers with existing views
Factor out details	Make views more specialized
Remove details not used	Make views more abstract
Remove details that are redundant	Extend views with information
Remove details that may be automated	Clean up views
Document details in code instead	Make views more formal
	Add more details to views
	Write better guidelines for views
	Make views easier to access
	Automate view generation

are invited as needed. Discussion outcomes are documented as issues to be resolved in succeeding analysis activities.

6.10. Improvement analysis (Q_3)

Improvement analysis is the analysis of a concern survey's qualitative answers, with the objective of explicating what increases and decreases informants would like to see. It complements concern plotting and zone mapping which prioritize needs, but say little about what to change. Improvement analysis proceeds by coding and categorization (cf., coding and categorization of concerns in Section 4.5), with the generic AD improvement categories of Table 5 as starting point. The categories emerged from the analysis activity in the Ericsson case; they capture (without going into details about specific formalisms, views, and information elements) what kinds of AD changes developers liked to see, hence also the action space in AD improvement. Decrease and increase categories represent the concrete suggestions that informants expressing a need for decrease and increase, respectively, have provided. (Notice that Table 5 shows categories of changes rather than concrete changes.)

To allow queries, survey answers are also coded with respect to the *area to which they belong*, whether they represent a suggested *increase or decrease*, what *kind of informant* that made the suggestion, the *project* for which the suggestions was made. This coding is entirely mechanical and can be automated in many modern survey- and data analysis tools.

Queries are used to produce the improvement analysis report (see Fig. 5). The report lists the concrete suggestions of the informants on condensed and organized form. It has a three-level structure: area of concern at the topmost level, kinds of increases and decreases at the level underneath, and the concrete needs (along with project- and informant information) on the lowermost level. The report is used both during decision making, where it provides details about what the “spots” in the concern plots stand for, and during improvement where it provides information on what the needs are and whom to interview for further details about the need.

Improvement analysis should be done by the analysts themselves (rather than being outsourced). Exposure to the actual needs, and the reflections triggered by the coding process, deepens the analysts' understanding on what lies underneath the statistics, which in turn improves the quality of the prioritization outcomes.

6.11. Key factor survey (Q_4)

Key factor survey is a technique to identify and assess factors with influence on architecture documentation practices that managers can relate to and are empowered and motivated to handle.

They Key factors are used to explain why the situation is as it is, and what managers can do to change the situation. Without attention to this problem, the actions proposed to managers and chief architects – the very outcome of our prioritization method – risks being perceived as “just another good idea” (Armour, 2011). The focus question is, thus, how architecture improvement proposals are best presented to management.

A key factor survey involves survey design and survey execution. Survey design proceeds by collaborative sessions around data displays; it results in a set of key factors and scales for assessing these. Survey design should involve software process improvement specialists and managers knowledgeable with improvement work within the organization. Discussions start with the generic factors and scales in Table 6 as input; it should include recognition and formulation of additional site-specific key factors, suggested by the study's findings so far. Concern plots are useful in this, as they exhibit patterns in data suggesting that certain forces are at play.

For instance, exposed to the concern plot of the area of logical concerns (see Fig. 12) and similar concern plots for other areas, a software process improvement specialist at Ericsson observed that concern areas with high degrees of documentation tended to be those important to project managers, while those with the lowest degree of documentation were those that project managers never had to deal with (the MSA factor in Table 6). Another observation in the plots were that concerns prevalent also in hardware had higher degree of documentation than those only present in software (leading to the HER and RES factors). It was observed that many of the concerns with weak support were difficult to model whereas areas with high degrees of documentation were often straightforward to model (the factor SIM). For some concerns, in particular in the area of concurrency, it was evident that the desired modeling notations were not supported by the modeling tools in use or that even the notation was an open question (the T&S factor).

Survey execution proceeds by the usual steps: informant selection, data collection, and statistical analysis. The survey should be sent to architects, technical managers, and other senior engineers in a position to understand the concern areas themselves, and with enough experience from the organization to be able assess management support and attention, heritage, qualities of modeling notations and tools, and practices in other disciplines than the own. For large organization, this may mean 15 years of engineering experience or more. Data collection simply means asking informants “How would you rate you organization is fulfillment of F” for all key factors in Table 6 (MSA, etc.), and “What are your reflections on these scales?” Statistics analysis involves generation of histograms and application of statistical tests to see if a reliable conclusion of the situation in the organization can be made on from the survey input.

6.12. Factor correlation

The factor correlation activity studies the relationships between key factors and the prioritization outcomes. The purpose is to communicate to managers that current architecture documentation practices are due to factors that managers can control – and should control if architecture documentation practices are to be improved. It proceeds by cross comparison of the key factor assessments for each area of concern, and the prioritization outcomes for these (i.e., whether their documentation should be increased, decreased, or left as it is).

An example for such a cross comparison is given by Fig. 11. Here, we see that areas desirable (INC) or important (INC+) to increase are associated with non-attention (MS0, 15 + 3) or lack of knowledge (MS–, 3 + 1): 23 (3 + 1 + 15 + 3) out of 24 (23 + 1) concerns in need for increase has not received management attention. Further, we see that 3 out of 5 (3 + 1 + 1), i.e. 60% of the concerns areas that do get high management attention (MS+) are over-documented

Table 6
Key factors that speak to decision makers, with scales to assess these.

MSA	HER	SIM	T&S	RES
Management support and attention	Heritage	Simplicity	Tools and standards	Resemblance
Degree to which managers care (or have cared) about these concerns:	For how long we have been working with these (or similar concerns):	Degree to which concerns are easy to articulate (regardless of tool, e.g. on paper):	Degree to which tools and standards support modeling of these concerns:	Degree to which SW documentation practices resemble those in other engineering disciplines:
MS+ = Some concerns in this area has (had) high attention and support from management, e.g., they occur in checklists, in priority lists, have been addressed by internal improvement activities, e.g., studies, focussed workgroups, new tool features, improvement projects, or pre-development.	HER+ = We have been documenting concerns in this area for decades, and have a longstanding tradition in documenting this.	SIM+ = These concerns are reasonably easy to understand: most engineers can relate to them and model them. Working with the views covering these concerns does not require special training. (This is not to say that the work in which the views are used is simple.)	TS+ = Concern in this area are recognized by and supported by the modeling tools available to the organization.	RES+ = This area is similar to what we also have in other engineering disciplines (e.g., hardware-, mechanical- or systems-engineering-concerns are documented similarly).
MS0 = Concerns in this area do not get much attention from managers. They are known by management, to some degree, but not considered important enough to spend money or time on.	HER0 = We have been documenting this in at least one project (> 1 year) and the concerns are clearly visible in documentation.	SIM0 = Work with concerns in this area requires specialist knowledge in some engineering domain: education in some theory or in the use of special tools, and many years of domain experience.	TS0 = Concerns in this area have been expressed with contemporary tools, but this has required in-house customization such as non-standard uses of notations, additional profiles, or addition of tool features.	RES0 = This area of concern resembles concerns in other engineering disciplines, but views are entirely different.
MS- = Most managers are probably not aware of concerns in this area, or their importance in SW development.	HER- = We have no tradition in documenting these concerns, at least not to the degree that it has affected downstream projects.	SIM- = Concerns in this area are very difficult to deal with: (at most) a handful of engineers within the project know how to work with these concerns well.	TS- = Concerns in this areas are not recognized by contemporary modeling tools and standards and do not quite seem to fit the UML paradigm.	RES- = This area of concern is software specific: it does not have corresponding concerns in other engineering disciplines.

(DEC). Together, these results suggest that managers should be more careful when directing the resources to architecture documentation work, i.e., they further motivate the research presented in this paper. They also suggest that efforts in moving management's attention away from what the organization is focusing on today, to make the organization actively look for concerns overlooked, would be an effective strategy in improving AD.

The factor correlation should also include statistical test to prove correlation. For instance, a standard correlation test on these data showed a significant negative correlation ($\rho = -0.46$) between management support (vertical axis in Fig. 11) and the importance of increasing documentation (horizontal axis in Fig. 11).

6.13. Inference making (Q_4)

The inference making activity of the last (Q_4) iteration verbalizes the findings from the factor correlation study using a vocabulary accessible to managers. The purpose is to express the needed improvements at a level that managers is accustomed to, while giving evidence that suggested improvement actions will improve architecture documentation in desired ways. The activity is carried out by the analysis team with involvement of managers, and proceeds by a case by case consideration of key factors. Each factor is turned in an action that will move fulfillment of the factor in the direction associated with documentation improvement. Other sources than correlations may also be used, such as survey data statistics.

The result is a set of *action principles* for management: succinct, empirically grounded sentences on active form that capture the essence of what managers should do to improve architecture documentation practice. For instance, the established relationship between management support and the importance of increasing documentation, along with survey statistics showing that many managers have weak intuitions about many concerns important to improve, led to formulation of the following action principles:

- Educate managers about architectural concerns
Avoid unconscious prioritizations due to lack of knowledge.
- Spend time on prioritizing concerns
Concerns are not equally important.

To be effective, the action principles should be compact enough to fit on a page, along with a reference to a report with the evidence that they are valid.

7. Example: concern prioritization outcomes of study at Ericsson

The following sections exemplify some of the outcomes of the concern prioritization activities of the Ericsson study. Again, outcomes are influenced by particular choices of analysis tools and presentation formats, and not a prescriptive part of the method. The business goal, G , in the prioritization was to *increase speed* i.e., to increase the organizations overall throughput, and to reduce the lead-time, for new features and change requests.

7.1. Concern plots (needs)

Concern plots were produced for each major area of concern (see Fig. 6). Fig. 12 shows the concern plot for the major area of logical concerns. The dots in the diagram represent concern areas whose definitions can be found in a technical report (Pareto, 2010); an outline of these areas and the specific concerns therein can be found in Table 3. We have chosen the logical concern area, for presentation in this paper, as it is the most richly populated, area and thus makes a good case for explaining our approach to prioritization.

		DEC-	DEC	OK	INC	INC+
Not known by managers	MS-	0	0	0	3	1
Known, but not prioritized by managers	MS0	0	4	4	15	3
Attention and support from management	MS+	0	3	1	1	0

Fig. 11. A comparison between the degree of management support for architectural concerns (MS), and the importance of decreasing (DEC) or increasing (INC) the documentation of these.

Interpretation of statistical displays, in general, does not lend itself well to operationalization. It is a reasoning activity in which data interacts with the analyst's understanding of the phenomena underlying the statistics; it depends on the analyst's reasoning skills and experience with statistics; it may, as a result, lead to several conclusions. This said, there are several routine comparisons that any interpretation should include: overall comparison of increase and decrease forces; point wise comparison of increase and decrease forces; consideration of extremes. We present, in the following, how Fig. 12 can be interpreted. The presentation follows the line of reasoning used in the Ericsson case.

7.1.1. Center of gravity

An overall eyeball comparisons of increase forces with decrease forces show that, increase forces are much stronger than decrease forces: the center of gravity of all points, except one (protocol realization), is clearly on the increase side of the diagram. Thus, as

far as the informants' overall opinions go, we should increase the documentation of logical concerns to increase speed.

7.1.2. Consensus

Point-wise comparison of increase and decrease forces shows consensus for one area only (Overview): here, nearly everyone wants to see increased documentation. For most other areas, there is a minority in favor of a decrease, except for Protocol realization, where the majority favors a decrease. Whatever prioritization we make, the areas with high consensus should be recognized and considered.

7.1.3. Distances

The distances between increase and decrease points (in the horizontal direction) indicate improvement need: high distances suggest a high need for improvement. Distance based ranking of

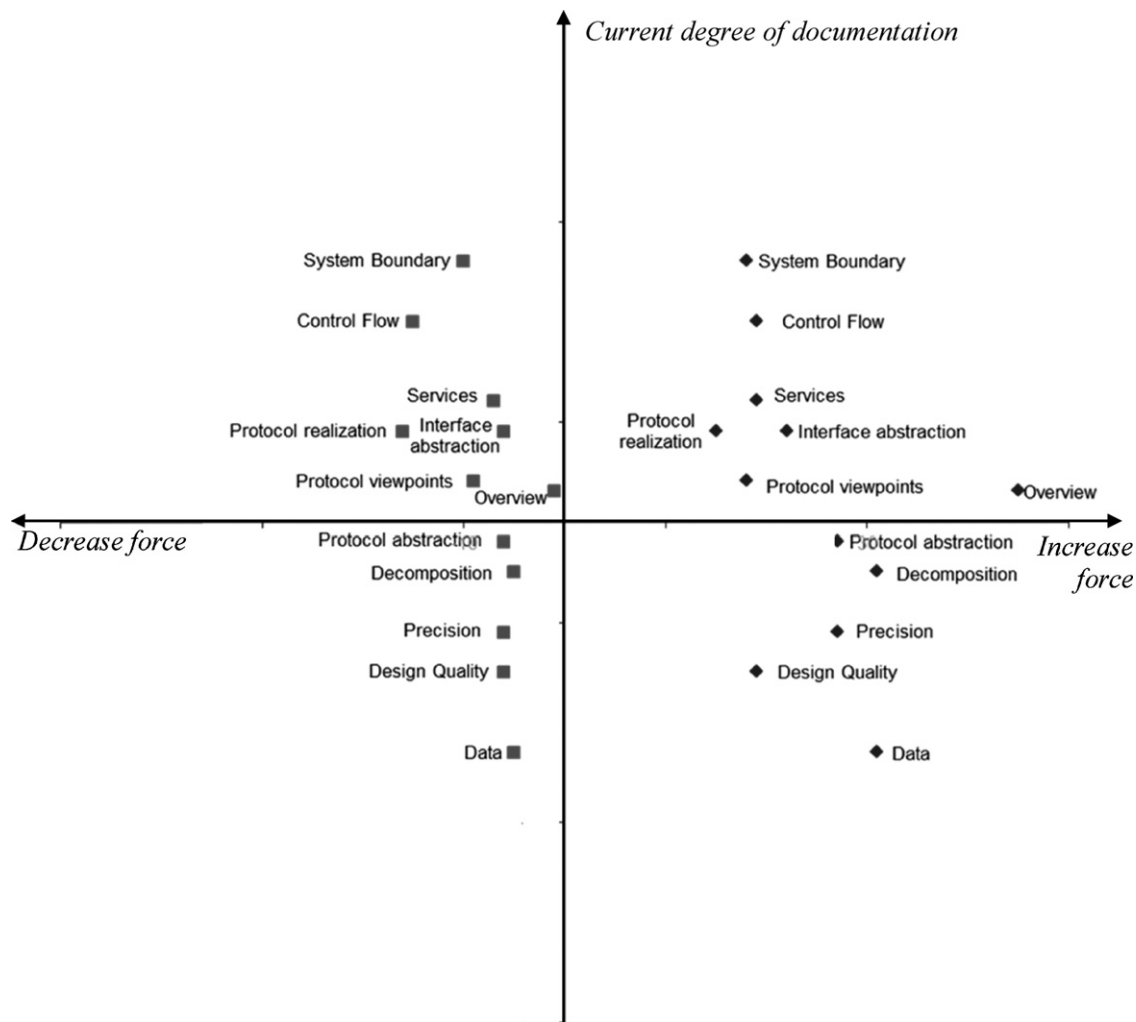


Fig. 12. Concern plot for the major area of logical concerns.

Table 7

Side by side comparison of Rankings (top), and the five areas most important to improve (bottom).

Consensus	Overview (1)
Distances	Overview (1), data (2), decomposition (3b)
Under documentation	Overview (1), data (2), decomposition (3b)
Extremes	Overview (1), system boundary (3c), data (2), protocol realization (3a)
Over documentation	Protocol realization (3a), control flow (4), system boundary (3c)
Top 5 improvement areas	Overview, data, protocol realization, decomposition, system boundary

the concerns in our diagram results in (1) overview-, (2) data-, (3) decomposition, as improvement candidates.

7.1.4. Extremes

Extremes along all axes point at areas in need for attention. Along the increase axis, we find Overview concerns to have the strongest increase force; along the decrease axis, we find Protocol abstraction to have the strongest decrease force; along the current degree of documentation, we see that data concerns have the lowest- and system boundary the highest values along the current degree of documentation axis. These four areas: (1) overview, (2) system boundary, (3) data, and (4) protocol realization should be considered for improvement, and looked into in greater depth.

7.1.5. Under-documentation

Low to moderate degrees of documentation combined with strong increase forces suggests that that an area is under documented and should be recommended for improvement. This results in (1) overview-, (2) data- and (3) decomposition to be considered for improvement.

7.1.6. Over-documentation

Symmetrically, moderate to high degrees of documentation combined with strong decrease forces should be given attention. This gives (1) protocol realization, (2) control flow, and (3) system boundary as improvement candidates.

A side-by-side comparison of our inferred rankings (see Table 7) allows prioritization. Overview concerns stand out as the most important area to improve: it appears as no. 1 in four out of five four rankings; data concerns are the next most important area, and appears among top 3 in three rankings; protocol realization, decomposition, system boundary are of roughly same importance. That is, the top 5 areas to improve are (1) overview, (2) data, (3) protocol realization, (4) decomposition, and (5) system boundary (possibly with permutation of items 3–5).

7.2. Improvements (needs)

The kinds and relative frequency of the suggested improvement are given in Fig. 13. Outstanding is the need for additional views, followed by more learning materials (introductions, etc.). The biggest class of suggestions to decrease documentation (formality-organizational) was not to decrease the documentation itself, but the bureaucracy surrounding it, such as approvals. Even this need was outnumbered by seven classes of suggestions for increased architecture documentation.

Some of the concrete suggestions for prioritized areas are given in Table 8.

7.3. Zone maps (priorities)

The prioritization resulting from zone mapping is given in Fig. 14. The areas most important to attend to are increased documentation for overview and data concerns (which are classified as

Table 8

Examples of architecture documentation improvements proposed by informants.

Area	Suggested improvements
Overview (logical concerns)	1. Architectural principle motivations. 2. More abstract views. 3. High level architecture overview. 4. Partitioning principles in different dimensions. 5. A clear overview. 6. Architectural description HowTo's.
Data (logical concerns)	1. Information on data. 2. Visibility of data propagation. 3. Data dependency diagrams.
Linkage (development concerns)	1. Traceability. 2. Automated traceability. 3. Design rules for build process. 4. Breakdown of requirements and traceability between black-box and white-box perspectives. 5. Relation between realization of managed objects and resource objects. 6. Dependencies and connections between data and protocols. 7. Traceable interaction from managed objects, to resource objects, to other parts.
Mode and phase specific concerns (domain concerns)	1. Guiding principles for mode and phase specific concerns. 2. Overview documentation for modes and phases covering multiple system levels. 3. Improved description of update and upgrade procedures. 4. Distinct separation of mode and phase specific concerns. 5. Informal documentation on mode and phase specific concerns when reasoning between disciplines. 6. Mode and phase specific overview information. 7. View of system restart. 8. Increased precision in description of mode and phase specific concerns.

INC+). Three concerns can advantageously be decreased (classified as DEC). Closest to the INC++ zone is in the zone map is overview, followed by data. Closest to the INC– zone is protocol realization. In this, we have given the areas on the decrease zap a “handicap” worth one zone.

A top 3 recommendation to management would thus be to fund improvement projects with the primary goal to improve documentation for overview (INC+), data (INC+), protocol realization (DEC). A top 5 recommendation would add system boundary (DEC) and control flow concerns (DEC) to the list.

7.4. Action principles (action)

The action principles (Table 8) are inferred from our data sources, supported by traceable inference making, and validated in a team workshop (Table 9).

7.5. Priority dashboard (action)

A statistical display particularly useful in communicating priorities to managers is the priority dashboard, which consists on the circular taxonomy (Fig. 4) but with priorities superimposed (see Fig. 15). Concern areas to the right in each major area (in the clockwise direction) are those most important to increase; concern areas to the left are those most important to decrease. Concern areas in the middle are OK, and do not need any attention.

7.6. Evidence (evidence)

Evidence for the action principles and improvement proposals is built from the *hypotheses* and *processed data* (statistical plots,

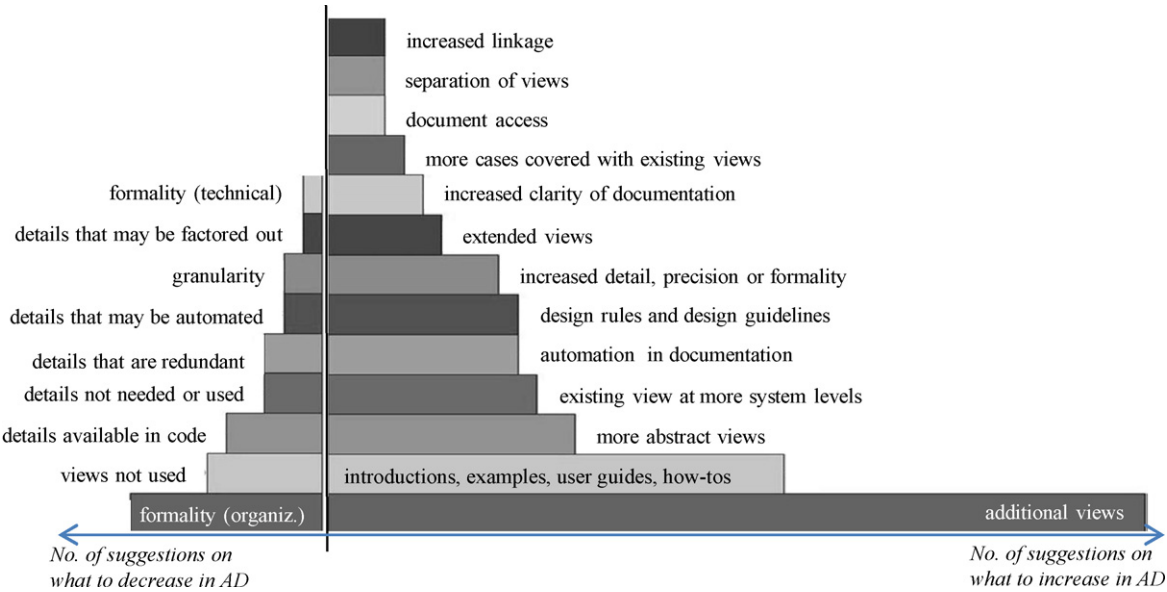


Fig. 13. Distribution of improvements found.

taxonomies, and tables) which in turn are built from *raw data* (survey answers, workshop notes, and transcripts). The evidence data in the Ericsson case amounts to roughly 100 pages, whereas raw data amounts to 300 pages. Data sources for the taxonomy are much larger: 43,000 words of transcripts from 16 h of interviews, 1440 pages of informal documentation, 217 screenshots, and 30 pictures of whiteboard drawings (Pareto et al., 2009).

8. Validity of results

Our methods are valid only if they can be effectively applied in a large organization, and produce results useful to and appreciated by the organization. This section describes the measures taken to produce a method that is valid (Sections 8.1 and 8.2), and the feed-

back from the studied organization on our method (Section 8.3) and on its outcomes (Section 8.4).

8.1. Validity through action research

The prioritization method was developed by an action research team consisting of three Ericsson insiders (a manager with a research background in software process improvement, a senior software specialist, and a senior software architect) and an outsider (a University researcher) situated at Ericsson during the whole project (2 days per week on average). Researchers were situated in the heart of the studied division, and had, in addition to the formal research activities, frequent informal contact with designers, architects, managers, and other roles in the division.

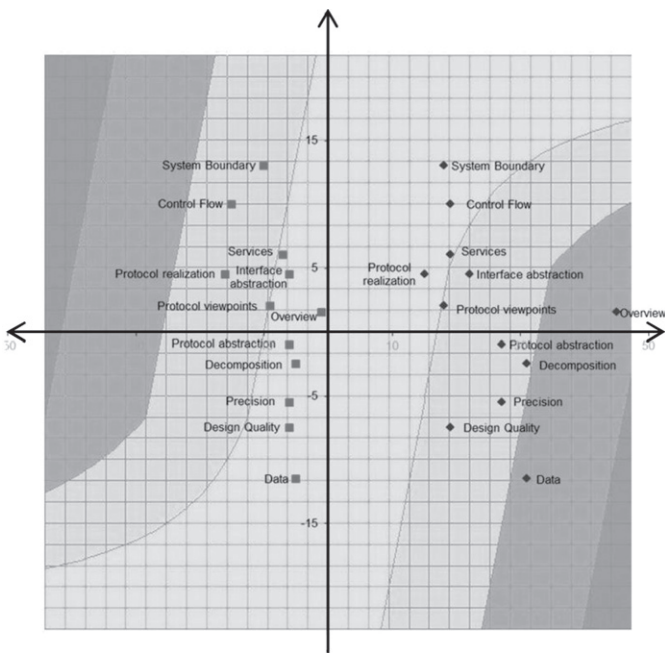


Fig. 14. The use of a zone map (left) to prioritize areas of concern (right) for the major area of logical concerns.

Logical concern areas	Dec	Inc	Priority	Att
System boundary	DEC	OK	DEC	⇐
Control flow	DEC	OK	DEC	⇐
Services	OK	OK	OK	
Interfaceabstraction	OK	INC	INC	
Protocol realization	DEC	OK	DEC	⇐
Protocol viewpoints	OK	INC	INC	
Overview	OK	INC+	INC+	⇐
Protocolsabstraction	OK	INC	INC	
Decomposition	OK	INC	INC	
Precision	OK	INC	INC	
Design quality	OK	INC	INC	
Data	OK	INC+	INC+	⇐

Table 9
Action principles for managers.

Target of change	Action
Designers	Encourage designers to share knowledge Designers know the needs: make sure to learn and share their views.
Architects	- Encourage architects to learn and share Architects must be willing to react on architecture improvement advice. - Move beyond out-of-the box modeling Don't be content with weak notations: utilize tools and extend. - Challenge the heritage The way you documented in the past is not the best way now. - Avoid the simplicity trap Don't prioritize concern areas just because they are simple.
Managers	- Educate managers about architectural concerns Avoid unconscious prioritizations due to lack of knowledge. - Reach beyond managers needs - Don't over-document in areas supporting managers' interests. - Spend time on prioritizing concerns Concerns are not equally important.

The division was extensively involved during data collection, in the informant workshops, in the team workshops, in the survey design, in the concerns catalogue formulation, in the concerns taxonomy formulation, in review meetings, and in the research writing processes. It provided access to more than 50 informants, many of which spent several days of engagement in interviews, in team meetings, as reviewers, in workshops, and as audience in project presentations.

Because the project involved several years of on-site work, and because researches spent extensive time on site, researchers developed understanding of the key forces and challenges in the

organization. This adds to the validity of the analysis, compared to an off-line analysis.

8.2. Validity through a grounded approach

The project followed a grounded theory approach (Strauss and Corbin, 1998). All artefacts in the method (concern catalogue, taxonomy, concern plots, and improvement list) are grounded in user data, with traceable connections that can be checked. Although this does not eliminate analyst bias, it reduces its influence.

8.3. Feedback on the method from the division

The method and its many parts received the following feedback from the division:

- Feedback on the *concern prioritization method*, from a senior specialist in software process improvement, was that data collection and analysis is more extensive compared to traditional prioritization practices, but that extensive analysis seems inevitable in architecture improvement work.
- Feedback on the *concern survey* by informants was that it was difficult, but that answering it lead to reflection on (and questioning of) current ways of working.
- Feedback on the *concerns catalogue* by informant workshop participants was that the concerns catalogue was a valuable checklist that would benefit from company-wide standardization, and that concern plots captured the needs well and were good for collective discussions of needs.
- Feedback on the concern prioritization method's *informant workshop* itself was that it led to reflection, and that it was very interesting to see and hear that individual reflections were consistent with those of peers.
- Feedback on the *action principles* was that although it is clear that more insight in some concern areas is needed, it is also clear that managers should not have to understand concerns to the same degree as designers do.

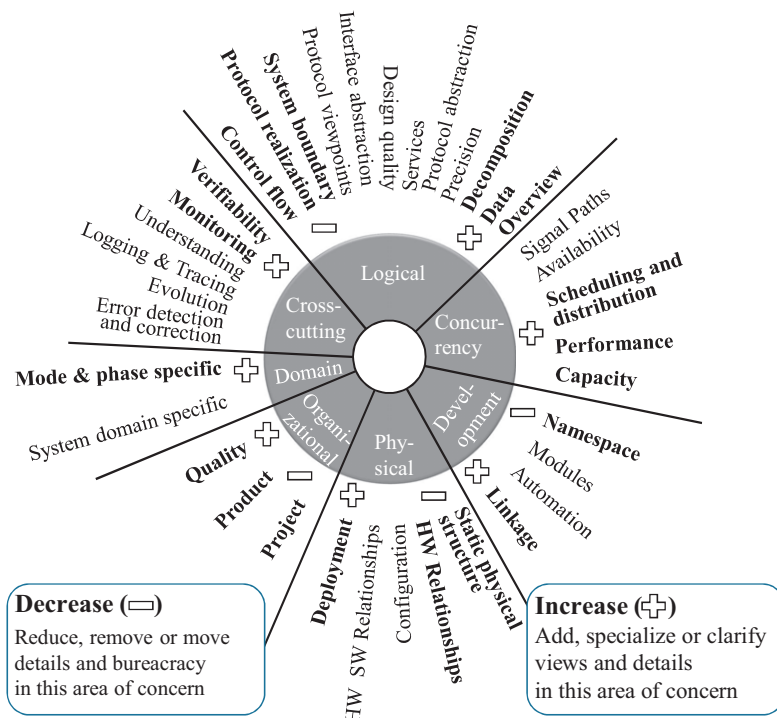


Fig. 15. Concern area priority dashboards.

- Feedback on the *concern plots* by a chief architect was that these were “highly useful”.
- Feedback on the *concern plots* by another chief architect was that “no one is doing this kind of user experience gathering” and that the architecture documentation, in much, is “open control” where it is actually very difficult for a chief architect to know if and how and to what degree architecture documentation is used.

8.4. Feedback on method outcomes (AD user feedback) from a chief architect

When exposed to the most prioritized increase, *Overview*, a chief architect approved that this improvement was indeed very important. The architect could immediately explain several causes of the problem: (1) that the modeling tool in use had a hierarchical process tree as dominating perspective; (2) that the tool had a high learning barrier and (3) that overview artefacts (informal diagrams) were not integrated with the modeling tool, i.e., pictures existed but could not easily be viewed from inside the modeling tool. The architect also had clear ideas about their resolution: to make AD reports as first class citizens in the software development processes; to implement viewpoint “filters” where a metamodel defines what the view should show, and where the actual overview is generated from the underlying more detailed model.

When exposed to the topmost need to decrease, *Control flow*, the architect reacted similarly: by immediately explaining what the underlying problem was (sequence diagrams used prescriptively rather than for elaboration, Statecharts underused in specifications, and resistance to throw away sequence diagrams) and how they could be resolved (distilling Statecharts from sequence diagrams, and throwing away sequence diagrams).

The architect’s reactions to these recommendations make it evident that the most prioritized problems, that our method produced, indeed were longstanding issues in the unit. It also showed that the architect knew how to solve these issues, but that he was either not empowered to implement the required changes or underfacilitated by his managers.

Together, this shows that our prioritization method’s approach, i.e., connecting user feedback to not only Chief architects, but also to upper managers that can facilitate Chief architects, is motivated. It also shows that the information provided to architects is comprehensible and useful.

9. Discussion of the method and its qualities

9.1. Prioritization towards a single business goal

The method relies on prioritization of documentation towards achieving a single business goal. It assumes that prioritization of business goals has already been resolved by upper management and that the organization is to realize this goal – a common situation in large product developing organizations. In the case presented in this paper, the business goal was increased speed, but it could have been any high level goal, such as increased quality, innovation, or efficiency. Optimization towards speed was an explicit requirement from Ericsson as optimization for, say, correctness could result in counterproductive improvements from the perspective of speed.

The method, as it stands, does not include techniques for negotiation of business goals, neither for establishing consensus around recommendations in case of conflicting interests among stakeholder groups. The method’s rationale is to support decision making through increased feedback and codification of current needs in the form of areas in most need for improvement (see Fig. 6). Methods

for decision making on the basis of this information is beyond the scope of the current paper, although such methods are needed too. The method can straightforwardly be extended to include collaborative business goal negotiation as a pre-activity and collaborative decision making of improvement projects as a post activity if that would be desired by an organization. It can also be used in (and is designed to support) centralized decision making on the basis of a collaboratively constructed recommendation.

9.2. Zone mapping versus inference making from concern plots

The zone map is primarily a communication device, as the rankings resulting from zone mapping, can easily be inferred directly from the concern plot and a few principles. We recall from Section 7.1 that such interpretation resulted in recognition of *overview*, *data*, *protocol realization*, *decomposition*, *system boundary* as areas to improve, which was the very same ranking that zone mapping resulted in. This is natural as the zone map relies on the same principles as our interpretations.

However, for the purpose of communicating results, the simplicity of zone maps makes them superior. The many reasoning steps involved in an interpretation of a concern plot may be difficult to follow, whereas it is easy to judge which point is closest to the curve for the next higher zone on a map. Using zone maps, it is also clear, that selection involves some degree of arbitrariness and that other factors should be allowed to influence recommendations made.

Thus zone maps are helpful, in supporting the recommendations made, and in communicating recommendation confidence. For instance, in the Ericsson case, overview concerns were outstandingly most important, followed by data concerns, whereas the internal order between protocol realization-, decomposition, and system boundary concerns was more arbitrary. Confidence in the first recommendation was also the highest.

9.3. Variability in tools used

Although the method description makes certain commitments to tools used (qualitative data analysis tools, survey software, and spreadsheets), tools should be considered as variation point in the method. More important than what tools are being used is that the organization is accustomed to them, e.g., a wiki can serve as data analysis tool.

9.4. Qualities of concern prioritization

A rating of our proposed method, with respect to the quality model for concern prioritization methods defined in Section 2, is given in Table 10. The rating is based on reflections on our experiences with the method and literature based comparison with other approaches to prioritization. We can see that the method scores well for most of the desirable qualities (involvement, interference, prerequisites, and accessibility), but not for cost.

9.5. Summary of experiences

When the project started, viewpoint engineering was carried out by a small team of senior architects that designed architecture documentations towards their (and their immediate networks) perceptions of the organization’s needs. Architects where, however, challenged by the problems of attending to the special needs of all communities, and consequently of getting acceptance for architecture descriptions and their principles across all stakeholder groups. Architects did actively listen to stakeholders, but the needs of designers, in particular, did not get the same degree of attention as they, from an economic perspective, would deserve.

Table 10

The degree to which the proposed concern prioritization exhibits qualities desirable for concern prioritization methods.

Quality	Rating
Cost	<i>Moderate to High.</i> Inventorying involves extensive analysis work over a prolonged period of time, giving high costs for establishment of taxonomy.
Involvement	Prioritization involves each informants spending at least one day on taking the survey and on workshops, as well as analysis work, and recurrent meeting for an analysis team. Costs for taxonomy may however be amortized over several replications of prioritization.
Interference	<i>High.</i> Key stake holders are involved in the data collection as well as in decision making. The use of surveys allows involvement of hundreds of stakeholders.
Prerequisites	<i>Low.</i> During inventorying, informants are only disturbed by a 1 h interview, and a workshop. During prioritization, informants are disturbed for ½ a day for taking the survey, and another ½ day for the informant workshop. Most of the work is done by a small analysis team, with occasional infrequent involvement of experts from within the organization.
Accessibility	<i>Low.</i> The activities involved resemble everyday management activities, and do not involve techniques requiring special training in statistics, or certain statistics or analysis tools. Ordinary office tools (spread-sheet software, survey collection systems, and wikis) are sufficient for all activities, including taxonomy building.
Rationality	<i>High.</i> The design of decision materials for managers and chief architects is user oriented. Materials are presented at a level of abstraction and on a form that these roles are accustomed to. The use of site specific vocabulary in the taxonomy and the recommendations makes decision support materials easy to comprehend.
Correctness	<i>Moderate.</i> Statistical displays and inference principles make the approach more rational than intuitive decision making; the interpretation of displays involves elements of intuitive reasoning, thus the approach is less rational than purely quantitative approaches.
Intuitivity	<i>High.</i> The method has extensive involvement of stakeholders as informants, in the informant workshop, giving recurrent opportunities for validation; data points in concern plots that do not appeal to intuitions of the informants receive strong critique, as do erroneous rankings.
Validity	<i>High.</i> Decision making is done by interpretation of complementary data sources, rather than by statistical tests.
	<i>Moderate to high.</i> Properly executed, the method has high validity. Thanks to many manual coding and reporting activities, it is however sensitive to human error. Member checking in informant sessions detect such errors, but there are no guarantees for this.

Experiences from our project are that codification of user experiences, although somewhat costly, can have important effects on ways of working. One effect of the study was awareness about the division being subject to “the tyranny of the dominating perspective” described by Ossher and Tarr (2001). What had been appropriate views in earlier phases of the product’s lifecycle had turned into a burden. The tyranny was caused by a UML 1.3 generation MDD tool, which led to an over-emphasis of hierarchical processes networks, state machines, and sequence diagrams. Many of the problems with the AD could be traced back to limitations of this tool. This awareness, in turn, triggered the start of an architecture definition project that, using an object model as main view, obtained much better support for the Overview concern. It also accelerated replacement of the tool with more modern technology. A third effect was recruitment of a full time position responsible for developing architectural views for concurrency concerns, which

our concern prioritization methods pointed out as in need for major improvement.

Another important experience is the appreciation of documentation. In times of Agile software development, we were surprised to find that a majority of the engineers held increased documentation as a way to increased speed. Speed is tightly associated with agile software development, which in turn is associated with reduced emphasis on documentation, see e.g., the Agile Manifesto (Beck et al., 2001). Apparently, the very informants asked (senior designers, architects, and managers, most of which had extensive experience with UML and all of which had long experience with software development in the large) were of the opinion that the road to enhanced speed was to increase rather than decrease the degree of architecture documentation.

10. Related work

Most works on prioritization in software engineering are concerned with product- or software requirements within a product development process (Berander, 2007; Berander and Andrews, 2005; Regnell et al., 1998, 2001; Chatzipetrou, 2010). Concern prioritization is different to classical requirements prioritization, because AD is not engineered in the same way as products are: processes for AD evolution are typically implicit; there is no organized elicitation of stakeholder needs, and decision procedures and economical frames are not as explicit. Requirement prioritization techniques address some challenges of concern prioritization, but do not provide a whole solution.

An early taxonomy for architectural concerns was Zachman’s framework for Enterprise Architecture (Zachman, 1987); a range of enterprise architecture frameworks has been developed since, e.g., TEAF, EAP, C4ISR, DoD Products Map, DoDAF, NIST Enterprise, DOE AR. Our taxonomy for base station concerns (Pareto et al., 2009, 2011) was influenced by the taxonomies of architectural concerns developed by Kruchten (1995) for software development in general, and Rozanski and Woods (2005) taxonomy for enterprise IT systems.

Medvidovic has studied concern coverage of architecture description languages (Medvidovic et al., 2007) with respect to needs.

A method with explicit focus on prioritization of architectural concerns has been presented by Clements et al. (2010); they present a three step method that involves a catalogue of architectural views typically needed by stakeholders, stakeholder workshops, internalization of informant needs by the architect, decision making by the architect, and feedback on decision making from informants. Our approach differs in its higher emphasis on analytical techniques and evidence building, its use of scalable enquiry instruments, and its recognition of economical decision makers as stakeholders in architecture improvement work. Other differences are that our method emphasizes improvement of existing AD on the basis of user experiences, whereas Clements et al. emphasize prioritization of concerns in the AD construction phases on the basis of generic best practices. Where Clements et al. emphasize prioritization of concerns in the AD construction phases; our method emphasizes evolution of existing AD. Our method also differs in its focus on organization of prioritization work, and in being goal agnostic.

Fairbanks (2010) proposes a risk driven approach to architecture, where risk perceived by designers, is used to prioritize what architecture is needed. Our method differs in its focus on how large scale, multiple stakeholder prioritization work may be organized. The improvement goal in Fairbanks’ method is good architecture (a local technical goal), whereas the improvement goal in our method is a global business goal, e.g., increased speed.

Koning and Vliet (2006) explore an alternative approach to architecture documentation improvement, where the stakeholder

relevance of the existing architecture documentation elements are assessed, and where improvement proceeds by refactoring AD into smaller stakeholder oriented chunks. The approach is different as it does not address prioritization per se, and through a more transformative than formative approach.

Poort and Vliet (2011) present a method based on architecting as a risk and cost management discipline. Core activities are risk- and cost-assessment during architectural analysis, cost and risk based ordering of concerns, concerns inventorying, emphasis on risk and cost in stakeholder communication, and cost- and risk oriented objectives. The method has been validated in several major industrial studies. The major difference to our approach is the emphasis on risks rather than stakeholder needs, and that the analysis is centralized rather than collaborative. Establishment of feedback loops is not part of the approach. Their method also relies on risk assessment as a critical skill that architects must possess whereas our approach is needs driven, and does not depend on such competence.

Closest to our approach, from a process perspective, is the distributed prioritization method of Regnell et al. (2001), targeted at market driven product development. This method is iterative, driven by a small team that enquires disparate social worlds using survey's, analyses the feedback using graphical statistical techniques, involves informant feedback, and arrives at high level requirements. It does however rely on internalization of the prioritization problem by the deciding managers, whereas, in our method, a team of analysts provides managers with decision support.

Another similar approach, from a process perspective, is the infrastructure renewal method of Karydas and Gifun (2006). The method is collaborative and proceeds by thematic group work sessions and extensive deliberation. This method, too, assumes that decision makers are extensively engaged in the prioritization activities over a prolonged time, whereas, in our method, analysts rather than decision makers do most of the work.

11. Future work

At the time of writing, findings from the study have led to implementation of model-based support for interface specifications (interface abstraction) and new design rules (design quality). Decisions have partly been based on the architecture advice and partly on factors not present in the prioritization. Future work involves comparing advice fed to the organization with actual evolution and to identify factors with influence on the decisions. Studies of the psychology behind preferences towards improvement by adding (rather than improvement by removal) and behind wanting more of what is there, i.e. documentation greediness, would allow for more precise surfaces of zone functions. Better calibration of the zones in the zone map so that the handicap is not needed would be an improvement. Quantification of the effect of greediness on answers would then have to be taken into account.

Another direction of research would be a compositional analysis of our method: what the qualities of the involved techniques are, with respect to the quality model of our conceptual framework, and how the qualities of the technique contribute to the qualities of the approach as a whole. This in turn could lead to introduction of stronger techniques for some activities. In the long run, this would lead to variation points for the method, allowing a company to make tradeoff between qualities through choice between simple (but inaccurate) or sophisticated (but more taxing) techniques, and similar tradeoffs.

An interesting phenomenon to study is why change sometimes does not happen even when it is clearly important.

On the technical side, algebraic formulation of the zone function would allow automated ranking and interactive adjustment of zone boundaries. Templates for inference making in various situations would make the approach simpler to apply.

Extending the method to involve also external stakeholders in AD (such as regulatory/standard certification agents and integrators), brings further challenges that may require additional activities for enquiry and deliberation. Other extensions would be to bring in techniques for requirements negotiation in the formulation of the business goal of the improvement, and in the decision making of which of the prioritized areas to actually start improvement projects for.

Effective viewpoint engineering must not only involve Inventorying and prioritization methods, but also methods for management of AD related decision, similarly to approaches in Software Architecture design decision management, e.g., that of Jansen et al. (2008) and Software Architecture knowledge management, e.g., that of Vliet (2008) and Farenhorst and Boer (2009).

It seems plausible that generalization of our method could be applied to prioritization problems in other domains than software architecture.

12. Conclusions

Evolving architecture documentation requires engagement at several levels of the organization: designers must articulate their concerns; architects must specify needed views; managers must provide economic resources and appropriate organization to manage the whole lifecycle of views. Core challenges in this are inventorying and prioritization of concerns, and effective communication of stakeholder needs to chief architects and managers.

We have presented a new method for concern prioritization for the purpose of viewpoint engineering in the context of architecture documentation evolution in product developing organizations. The method combines collaborative and analytical techniques, and engages multiple stakeholder groups to produce grounded recommendations of what areas of architecture documentation to improve, and what to improve in these areas.

The method is the result of three years of action research in a division of Ericsson developing base station software. The method has been developed to meet needs as they arise in a real situation; the outcome of the method has been incrementally validated throughout its development. Recommendations have been well received by the organization studied.

Major characteristics, compared to the other methods are user-need based (rather than risk-based) prioritization, a transformative rather formative approach to architecture documentation improvement, the use of collaborative decision making, and emphasis on feedback loops.

The objective of the research presented in this paper is a method for prioritization of architectural concerns held by architectural documentation stakeholders within enterprise-scale product developing organizations. Conventional approaches to requirements prioritization are not suited to the problem, as the entity specified (architecture documentation) as well as its role in the product developing organization, and the processes surrounding it, are different. Desirable qualities of a concern prioritization method are low cost, high involvement of stakeholders, low interference with the daily work, high accessibility of method outcomes, high rationality in resulting priorities, high degrees of intuitive decision making, high correctness, and high validity. The method presented exhibits most of these qualities; the method's weakest area is cost, at the level of 3 man-months per year, and stakeholder involvement at the same level. The method is an important step in viewpoint engineering – the systematic application of engineering methods to assure the fitness for purpose of architecture documentation in

all usage situations. However, in large product developing organizations, such as those of Ericsson, where several hundreds of developer depend on architecture documentation in their daily work, the savings that can be made from improved architecture documentation are clearly large enough to outweigh the costs for viewpoint engineering, which, with appropriate methods and people, only involves a handful of engineers.

Acknowledgements

Research supported by Ericsson Software Research through Ericsson's Software Architecture and Quality Centre (SAQC). Thanks to the many informants at Ericsson that engaged in the study, and the anonymous reviewers for insightful critique of earlier versions of this paper.

References

- ANSI/IEEE Std 1471:: ISO/IEC 42010 – Systems and Software Engineering – Recommended Practice for Architectural Description of Software-Intensive Systems, 2007.
- Armour, P.G., 2011. Don't bring me a good idea. *Communications of the ACM* 54 (1), 27–29.
- ATLAS.ti Scientific Software Development GmbH, 2012. Atlas.ti – Qualitative Data Analysis. Available from: <http://www.atlasti.com>.
- Beck, K., et al., 2001. Manifesto for Agile Software Development, Available from: <http://agilemanifesto.org/>.
- Berander, P., 2007. Evolving prioritization for software product management. In: Department of Systems and Software Engineering School of Engineering, Blekinge Institute of Technology, Sweden, Blekinge.
- Berander, P., Andrews, A., 2005. Requirements prioritization. In: Engineering and Managing Software Requirements. Springer-Verlag, Heidelberg.
- Börjesson, A., 2006. Making Software Process Improvement Happen. Department of Applied IT, IT University of Gothenburg, Gothenburg.
- Börjesson, A., Mathiassen, L., 2004. Successful process implementation. *IEEE Software* 21 (4), 36–44.
- Chatzipetrou, P., et al., 2010. Prioritization of issues and requirements by cumulative voting: a compositional data analysis framework. In: 36th EUROMICRO Conference on Software Engineering and Advanced Applications. IEEE Computer Society.
- Christensen, H.B., Hansen, K.M., Schougaard, K.R., 2008. Ready! set! go! An action research agenda for software architecture research. In: Seventh Working IEEE/IFIP Conference on Software Architecture, pp. 257–260.
- Clements, P., et al., 2010. Documenting Software Architecture – Views and Beyond. SEI Series in Software Engineering, 2nd ed. Addison Wesley, Boston, MA.
- de Boer, R., van Vliet, H., 2011. Experiences with semantic wikis for architectural knowledge management. In: Working IEEE/IFIP Conference on Software Architecture (WICSA). IEEE Computer Society, pp. 32–41.
- Dickens, L., Watkins, K., 1999. Action Research: Rethinking Lewin, pp. 127–140.
- Fairbanks, G., 2010. Just Enough Software Architecture – A Risk Driven Approach. Marshal and Brainerd, Boulder, CO.
- Farenhorst, R., et al., 2009. The Lonesome Architect. In: Proceedings Joint IEEE/IFIP Working Conference on Software Architecture & European Conference on Software Architecture (WICSA/ECSA 2009). IEEE Computer Society, pp. 61–70.
- Farenhorst, R., de Boer, R.C., 2009. Knowledge management in software architecture: state of the art. *Software Architecture Knowledge Management: Theory and Practice*, 21–38.
- Jacobson, I., 1992. Object Oriented Software Engineering: A Use Case Driven Approach. Addison-Wesley.
- Jacobson, I., 1987. Object-oriented development in an industrial environment. *ACM SIGPLAN Notices* 22 (12), 183–191.
- Jansen, A., Bosch, J., Avgeriou, P., 2008. Documenting after the fact: recovering architectural design decisions. *Journal of Systems and Software* 81 (4), 536–557.
- Kaaränen, H., 2005. UMTS Networks: Architecture, Mobility, and Services. New York, Wiley.
- Karydas, D.M., Gifun, J.F., 2006. A method for the efficient prioritization of infrastructure renewal projects. *Reliability Engineering & System Safety* 91 (1), 84–99.
- Koning, H., Vliet, v.H., 2006. Real-life IT architecture design reports and their relation to IEEE Std 1471 stakeholders and concerns. *Automatic Software Engineering* 13, 201–223.
- Kruchten, P., 1995. Architectural blueprints – the “4+1” view model of software architecture. *IEEE Software* 12 (6), 42–50.
- Lewin, K., 1946. Action research and minority problems. *Journal of Social Issues* 2 (4), 34–46.
- Linden, F.v.d., Schmid, K., Rommes, E., 2007. Software Product Lines in Action: The Best Industrial Practice in Product Line Engineering. Springer, Berlin, New York, xx, 333 pp.
- Medvidovic, M., Dashofy, E.M., Taylor, R.N., 2007. Moving architectural description from under the technology lamppost. *Information and Software Technology* 49 (1), 12–31.
- Miles, M.B., Huberman, A.M., 1994. Qualitative Data Analysis – An Expanded Sourcebook. SAGE Publications, Inc.
- Ossher, H., Tarr, P., 2001. Using multidimensional separation of concerns to (Re)shape evolving software. *Communications of the ACM* 44 (10), 43–50.
- Pareto, L., 2010. Architectural concerns in base station development. In: Research Reports in Software Engineering and Management. University of Gothenburg, Gothenburg.
- Pareto, L., Boquist, U., 2006. A quality model for design documentation in model-centric projects. In: Proceedings of the 3rd International Workshop on Software Quality Assurance. ACM, Portland, Oregon.
- Pareto, L., Eriksson, P., Ehnebm, S., 2009. Concern visibility in base station development: an empirical investigation. In: 12th Intl. Conference on Model Driven Engineering Languages and Systems (MODELS 2009). Springer, Denver, CO, USA.
- Pareto, L., Eriksson, P., Ehnebm, S., 2010. Architectural descriptions as boundary objects in system and design work. In: Model Driven Engineering Languages and Systems, Pt II, pp. 406–419.
- Pareto, L., Eriksson, P., Ehnebm, S., 2011. Concern coverage in base station development: an empirical investigation. *Software and Systems Modeling*, <http://dx.doi.org/10.1007/s10270-010-0188-2>.
- Poort, E., Vliet, H., 2011. Architecting as a risk- and cost management discipline. In: 9th Working IEEE/IFIP Conference on Software Architecture (WICSA). IEEE Computer Society, pp. 2–11.
- Regnell, B., Beremark, P., Eklundh, O., 1998. A market-driven requirements engineering process: results from an industrial process improvement programme. *Requirements Engineering* 3 (2), 121–129.
- Regnell, B., et al., 2001. An industrial case study on distributed prioritisation in market-driven requirements engineering for packaged software. *Requirements Engineering* 6 (1), 51–62.
- Rockstrom, A.S., 1982. SDL-CCITT specification and description language. *IEEE Transactions on Communications* 30 (6), 1310–1318.
- Rozanski, N., Woods, E., 2005. Software Systems Architecture: Working with Stakeholders Using Viewpoints and Perspectives. Addison-Wesley, Upper Saddle River, New Jersey.
- Saaty, T.L., 1990. How to make a decision: the analytic hierarchy process. *European Journal of Operational Research* 48 (1), 9–26.
- Sandberg, A., Pareto, L., Arts, T., 2011. Agile collaborative research: action principles for industry-academia collaboration. *IEEE Software* 28 (4), 74–83.
- Star, S., Griesemer, J.J., 1989. Institutional ecology, ‘translations’ and boundary objects: amateurs and professionals in Berkeley's Museum of Vertebrate Zoology, 1907–39. *Social Studies of Science* 19 (3), 387–420.
- Strauss, A., Corbin, J., 1998. Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory. SAGE Publications Inc., Thousand Oaks, CA.
- Susman, G.I., Evered, R.D., 1978. An assessment of the scientific merits of action research. *Administrative Science Quarterly* 23 (4), 582–603.
- Tomaszewski, P., Berander, P., Damm, L.O., 2008. From traditional to streamline development – opportunities and challenges. *Software Process: Improvement and Practice* 13 (2), 195–212.
- van Vliet, H., 2008. Software architecture knowledge management. In: Aswec 2008: 19th Australian Software Engineering Conference, Proceedings, pp. 24–31.
- Zachman, J.A., 1987. A framework for information-systems architecture. *IBM Systems Journal* 26 (3), 276–292.



Lars Pareto is senior lecturer in software engineering and management in the department of computer science and engineering at Chalmers – University of Gothenburg. Lars received a M.Sc. in Computer Science and Engineering in 1992 and a Ph.D. in Computing Science in 2001 from Chalmers. Lars has conducted collaborative research with Ericsson AB since 2006 and published several studies related to modeling based software engineering. Lars teaches in model driven development, object oriented analysis and design, and model driven engineering, at the Software engineering programs in Gothenburg. He is a member of the ACM.



Anna Börjesson Sandberg is the head of process, methods, and tools for Ericsson AB's Design Unit for Internet Protocols and Broadband group. Her research interests include lean and agile ways of working, software process improvement, and collaborative practice research. Sandberg received a Ph.D. in applied IT from the IT University of Gothenburg, in 2006. She has published several studies on software process improvement activities at Ericsson AB. Anna is a member of IEEE and the ACM.



Peter Eriksson is a senior specialist in software architecture and real-time distributed systems, at Ericsson AB. Peter received his M.Sc. in electrical engineering from Chalmers University of technology in 1981, and has been affiliated with Ericsson AB since then. Peter has over 25 years experience of software development as programmer, designer, architect and process engineer, in the context of real-time systems for radio base stations. Peter contributed to the introduction of model driven development in this context, and is engaged in Ericsson's long term strategic work for software architecture, modeling, and tool development.



Staffan Ehnebom is a senior specialist in software modeling at Ericsson AB. He received a Master in electrical engineering from Chalmers in 1975, and has been affiliated with Ericsson since then. Staffan has ≥ 30 years experience as programmer, designer, architect and process engineer in the areas of real-time systems and information systems. He was a key contributor to Ericsson's TMOS SDPM – one of the first process models to comprise iterative & incremental software development. Staffan is acknowledged in Jacobson, *Object-oriented Software Engineering – A Use case Driven Approach* (1992), and Jacobsson, Griss, Jonsson, *Software Reuse* (1997).